MC SEM 2 Project Report — BEng Group 9

S. O. B. — Solar Optimizing Blossom



Written by: *Ana-Cristina Calugar, Anne Cecilia Augustinsson, Bianka Rinézer, Iustinia-Vera Butescu, Kiarash Zara Nasrabadi, Ridhima Rastogi, Sevgin Luis Novais Emilov*

Date of Submission: 02. 06. 2025.

Abstract

Modern security systems often rely on conventional power sources, making them both less sustainable and more conspicuous. As environmental awareness grows, there is a clear demand for compact, energy-efficient, and eco-friendly security solutions. Our project responds to this by developing **S.O.B. (Solar Optimizing Blossom)** - a self-sustaining, solar-powered security device disguised as a decorative flower.

Table of Contents

Abstract....1
Table of Contents....1
1. Introduction....2
2. Problem Formulation....2
2.1. Problem Introduction....2
2.2. Problem Statement....2
2.3. Project Scope and Objectives....3
2.4. Challenges & Considerations....3
3. Design Process....3
3.1. Early Development and Conceptualization....4
3.2. Component Design and Manufacturing....4
3.3. Visual Appeal Implementation....10
4. Electrical and Software Tasks....11
4.1. Solar Panels....11
4.2. Servo Motor....14
4.3. Battery and Power Supply....19
4.4. Motion Sensor....21
4.5. Nextion Display....23
4.6. Speakers....27
4.7. LDRs....34
5. Final Code and Assembly....36
5.1. Overview of Final Product Function....36
5.2. Construction and Circuit Assembly....45
6. Project Evaluation and Conclusion....46
6.1. Other Challenges and Learning Outcomes....46
6.2. Evaluation of the Fulfillment of Requirements....48
6.3. Reflection on Project Group Dynamic and Collaboration....52
7. References....54
8. Appendix....56
8.1. Machine and Technical Drawings....56
8.2. Separate C Codes....65

1. Introduction

The aim of this report is to provide an overview of the development process of a self-sustaining, solar-powered security device as part of our Semester Project 2. Additionally, the report provides a thorough evaluation on the project outcome and reflections on possible future improvements. The contents of this report are structured into chapters based on different aspects of the project. Following the *Introduction*, which serves as *Chapter 1*, *Chapter 2* is a brief summary of our initial *Problem Formulation* and objectives. After this, *Chapter 3* discusses the *Design Process* which is followed by an account of the *Electrical and Software*, as well as the *Mechanical Tasks* in *Chapters 4 and 5*, respectively. At the end of the report, in *Chapter 6*, we reflect on the *Challenges, Learning Outcomes* and the overall quality of our *Teamwork*. *Chapter 7* is a comprehensive list of our *References*, while Chapter 8 includes our *Appendix*.

2. Problem Formulation

In *Chapter 2* we included our original Problem Formulation for the Solar Optimizing Blossom (S.O.B.). Project objectives, requirements and other considerations are discussed in detail.

2.1. Problem Introduction

Security systems are essential for protecting homes, businesses, and private properties. However, many traditional security solutions rely on external power sources, making them less sustainable and often conspicuous. With growing environmental concerns, there is a demand for eco-friendly, energy-efficient security solutions. Our project addresses this need by developing a compact, portable security system powered entirely by solar energy.

Our product features a decorative flower-shaped design, where solar panel petals collect and store sunlight. This energy powers a motion sensor system that activates a dog-barking sound when movement is detected. To optimize energy collection, servo motors adjust the solar panel petals throughout the day based on input from photo sensors that track the sun's position. This approach ensures a self-sustaining and aesthetically pleasing security system.

2.2. Problem Statement

The core objective is to design a self-sustaining, solar-powered security system that meets the following key requirements.

To ensure our product meets expectations, we defined the key requirements and testing methods. Under Duty Requirements, we explain the requirements that the product must absolutely fulfill, while within the Optional Requirements, we list the additional goals we set for ourselves which are not crucial to the operation of our product.

2.2.1. Duty Requirements

- **Energy Management & Storage:** The system should function using solar energy during the day and stored energy at night (approx. 9 hours). We will select suitable solar panels and batteries through market research.
- **Motion Detection:** The sensor should detect movement within 3–5 meters. Testing will involve real-world trials and manufacturer specifications.
- **Solar Panel Adjustment:** Panels should maintain a 90° angle to the sun. A tracking system in our code will measure and adjust panel positions.
- **Autonomous Operation:** The system should function independently after installation, requiring only occasional maintenance.
- **User-Friendly Display:** The Nextion display should be simple and intuitive. We will test usability with a group of individuals.
- **Built-in Speaker:** The alarm should reach 70-80 dB. Testing will use either a sound meter or human perception.
- **Compact & Decorative Design:** The system will be flower-shaped and weigh around 2 kg for portability.
- **Budget:** The total cost should not exceed 2000 DKK.

2.2.2. Conditional Requirements

- **False Trigger Detection:** If feasible, we may implement a system to distinguish real threats from small animals or environmental changes.
- **Low-Power Mode:** To extend battery life, components should enter sleep mode when inactive.
- **Visual Appeal:** To enhance customer satisfaction, we would like to design a display that is not only functional, but also provides a pleasant user experience.

By following these guidelines, we ensure our product is functional, efficient, and practical.

2.3. Project Scope and Objectives

- Develop a motorized system with sensors to track sunlight.
- Store energy efficiently for practical applications.
- If feasible, integrate a security feature that activates upon motion detection.

2.4. Challenges & Considerations

- **Energy Storage:** Limited battery capacity may affect nighttime operation, especially in winter.
- **Security Effectiveness:** The artificial barking may not deter intruders.
- **False Alarms:** Small animals may trigger the system, causing disturbances.

Despite these challenges, we aim to refine and optimize our design.

3. Design Process

This chapter focuses on the more creative aspects of our project. The original idea, its practical execution and the implementation of some of our conditional requirements will be discussed in detail.

3.1. Early Development and Conceptualization

We began our project with a series of brainstorming sessions to figure out what kind of product we wanted to create. The main requirement from our supervisors was that the project should involve servo motors, but beyond that, we had the freedom to be creative. So, we started by doing some research - looking up projects online, watching videos, and reading about what other students or hobbyists had done with servo motors.

We came across a wide range of ideas: from robotic arms to automatic window blinds, vintage clocks, and even simple moving toys! One project, however, that really stood out was a flower-shaped device that used servo motors to move its petals and follow the sun. Even though using solar panels was not required, we thought it would be a great addition for our project - it would make our project more sustainable and allow it to run without needing a constant source of external power supply.

That is when we decided to design something like a “smart flower” - but with a twist. We thought, why not make it useful as a security system too? So we came up with the idea of a decorative, solar-powered flower that tracks the sun and reacts to motion by playing a dog barking sound to scare off intruders. This way, we were using the servo motors in a dynamic, purposeful way, while also creating a project that was both functional and fun.

Everyone in our group was involved in the creative process. We all had different ideas and ways of thinking, but we worked well together and built on each other’s suggestions. Some of us were focused on the technical side - thinking about electronics and components - while others were more into the design and overall concept. We sketched ideas, discussed possibilities, and looked at the challenges from different angles.

Even when we did not agree at first, we always managed to talk things through and come up with the best possible solution as a team. This open-minded and collaborative approach helped us land on a unique and well-rounded idea: the **Solar Optimizing Blossom (S.O.B.)** - a creative, self-powered security flower that shows how engineering can be both useful and beautiful.

3.2. Component Design and Manufacturing

3.2.1. Servo Motors and Motor Holders

I. Design Overview

Our initial design positioned the servo motors at the top of the stem, centered within the flower structure. In this setup, a fixed servo motor mounted on the stem was intended to control horizontal (X-axis) movement by rotating a second servo motor. The second servo, connected to the solar panels, would then provide vertical (Y-axis) motion.

However, upon receiving the motors and doing some preliminary testings to finalise our design, we found them to be too large and heavy to mount securely at the top of the

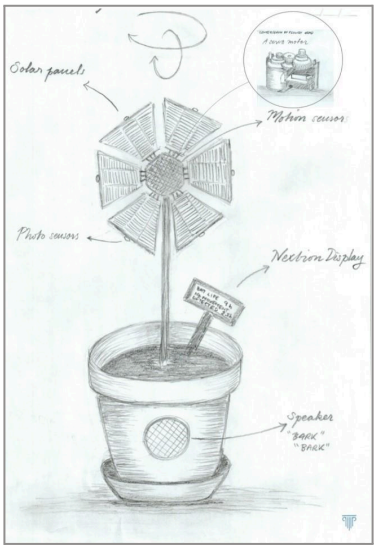


*Fig 3.1 - Initial Design

stem. For the purposes of stability and a more bulletproof design, we decided to relocate the servo motors to the base of the stem - housed within the pot.

The new issue now was how to move the solar panel using the servo motors now that there was so much distance between them. For this, we brainstormed a few ideas. One of them was a belt-driven system that would use flexible belts to transfer motion. It was lightweight, but needed precise tensioning and potentially could slip over time. Another idea was to use a gear and shaft system. However, we ruled this out because, although reliable, it was simply too heavy and complex for our needs. Keeping the motors at the top with long wires did not solve the stability problem.

In the end, we introduced a pulley system consisting of strings that are connected to the solar panels, allowing the servos to control their movement from below. The pulley system was the best choice because it's light, easy to build, and reliable, keeping our solar tracker stable and effective. The technical drawing for our design could be found in *Appendix 8.1*.

II. Servo Motors

Our initial focus was on integrating servo motors, as they were the central component and core function of our project. The design required a stable mounting structure to hold the motor securely within the housing, while also connecting it to a potentiometer to establish a feedback loop for measuring rotational angle.

One of the early challenges we faced was the motor's high RPM, which was unsuitable for our application. To address this, we used a motor driver along with a gear reduction system to lower the speed and increase control precision.

We used 10 gears in total to reduce the speed of the motor and the gear ratio calculation is made below:

- **Gear 1** → **Gear 2**: 48 / 24 = **2.0**
- **Gear 3** → **Gear 4**: 48 / 12 = **4.0**
- **Gear 5** → **Gear 6**: 48 / 12 = **4.0**
- **Gear 7** → **Gear 8**: 48 / 12 = **4.0**
- **Gear 9** → **Gear 10**: 48 / 12 = **4.0**

Total Gear Ratio = 2.0 · 4.0 · 4.0 · 4.0 · 4.0 = **512**

This is the highest Gear ratio we could make based on our design and application, otherwise either the gears would be bigger with higher number of teeth or we had to mount more gears on top of eachother.

$$\textbf{Output RPM} = \frac{Motor\ RPM}{Gear\ Ratio} = \frac{255}{512} = \textbf{0.4980 RPM}$$

$$\text{Time per rotation} = \frac{60}{\text{RPM}} = \frac{60}{0.4980}$$

$$\text{Time per rotation} \approx 120.48 \text{ seconds}$$

$$\text{Time for } 300^\circ = \frac{300}{360} \times 120.48 \approx 100.4 \text{ seconds}$$

With an output of approximately 0.4980 RPM, our geared system completes one full rotation every 2 minutes. While this may seem modest, it is appropriately matched to the needs of our application. Each day, the system executes a single sweep of up to 300° per motor during the initial alignment phase, which takes = 1 minute and 40 seconds. After this, only minor adjustments are needed throughout the day to keep the panels optimally aligned, minimizing overall movement and conserving energy. Our current power setup - two 3.7V batteries in series, supplying under 7.4V to the motors, Arduino Nano, and LDR sensors - is sufficient for this performance, especially since other components like the motion sensor and speaker are inactive during alignment. Future upgrades to the power system, such as adding a third Li-Ion battery for a 9-10V supply, will increase motor speed. However, the high gear ratio will continue to ensure ample torque and a controlled, steady rotation that aligns well with our structural requirements.

In terms of structural implementation, the first version of our servo motor housing was a single-piece design. However, this caused fitment issues, as the motor was obstructed by the gear mounting plate. To resolve this, we redesigned the enclosure into two separate parts: the top section, which held the gears and potentiometer, and the bottom section, which housed only the motor.

Initially, we planned to mount the potentiometer on the lower part, but we encountered a mechanical problem. The gear connected to the potentiometer had a long, thin shaft that was prone to bending or breaking under frictional stress. To mitigate this, we relocated the potentiometer to the top section for better support and durability.

The final component of the servo mechanism was the output gear, which engaged with the potentiometer to adjust its angle as the motor rotated. Our initial design featured a single tooth that slotted into the potentiometer's gear gap. However, this proved unstable; under rotation, the gear would easily tilt and disengage. We resolved this by developing a circular coupler that fully enclosed the potentiometer shaft, providing improved grip and preventing misalignment during operation.

III. Motor Brackets

After completing the design of both servo motor housings, the next step was to create a structure that could securely hold the two servo motors stacked vertically, while also allowing the stem to be attached.

Our initial concept involved a four-legged support structure. However, we quickly realized that the servo motors were not perfectly square but rectangular, which made the four-legged design less effective in providing a snug and stable fit. We then explored a three-legged design, but after multiple iterations and test prints, it also proved to be unstable and did not adequately secure the motors.

At this stage, we reconsidered the original four-legged approach. To improve its effectiveness, we modified the design so that the fit around the servo motors was very tight. This ensured that even if only two of the legs made direct contact with the motors, they would still hold them firmly in place. The remaining two legs functioned as additional support, preventing any unwanted movement or displacement during the operation.

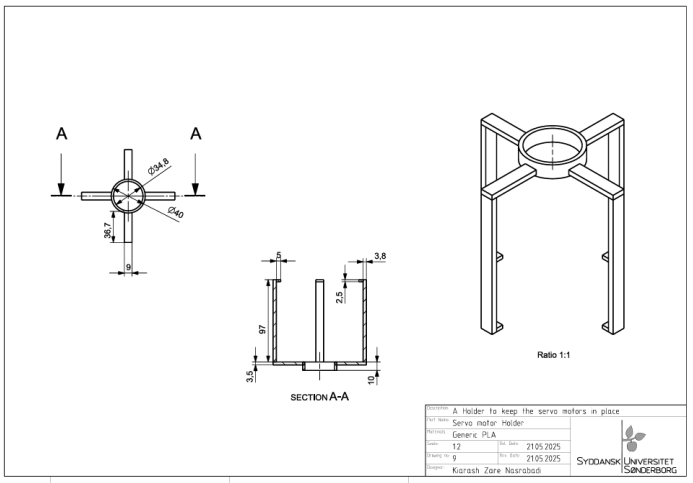


**Fig 3.2 - Holder to keep servo motors in place*

3.2.2. Flower Anatomy - Pot, Stem and Flower

I. Pot

The pot was the simplest in terms of design complexity but the most time-consuming to print due to its size and integration requirements. It needed to be large enough to house all internal components, including the two servo motors.

In addition, precise measurements were necessary to ensure the correct tolerances for mounting external elements such as the motion sensor and speaker, allowing them to extend slightly from the pot while remaining securely fixed in place.

To ensure a secure fit for the top cover of the pot, we measured the combined height of the two stacked servo motors, adding a few centimeters for clearance. Using this height, we determined the appropriate internal diameter of the pot at that level to achieve a snug fit and prevent the support structure from slipping downward. Once the dimensions were finalized, the support platform was laser cut with precise tolerances. Afterward, we manually drilled mounting holes based on the positions of the Arduino, motor driver, and battery holders to allow for proper assembly and alignment. Rather than spending additional time taking exact measurements of each individual component during the design phase, we opted to manually drill the mounting holes afterward. This approach was significantly quicker and allowed us to make real-time adjustments. Importantly, it also gave us the flexibility to experiment with the

physical placement of components like the Arduino, motor driver, and battery holders, enabling better decisions based on the overall visual appeal and layout of the product.

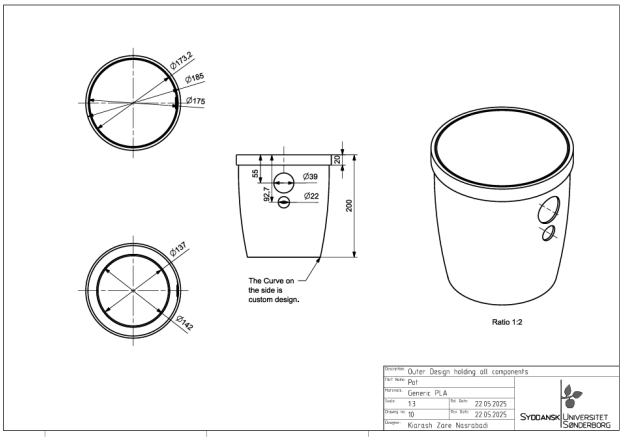


**Fig 3.3 - Outer design holding all components*

II. Stem

The stem was the second major component of our design and served two primary functions: supporting the solar panels and concealing the internal wiring and strings connecting the servo motors and other components to the panels.

A significant challenge was designing the ball joint at the top of the stem, which needed to fit precisely into the socket. We carry out multiple iterations of 3D printing and testing to achieve the right balance - ensuring a snug fit while maintaining smooth rotational movement with minimal friction.

In the initial design, the stem was made too thick, allowing for the passage of over 30 wires. While this provided ample space for internal routing, it appeared disproportionate relative to the rest of the structure, making the overall design look unbalanced. We revised the design by reducing the stem diameter to better match the scale of the other components. Although this limited the internal space, we required only 20 wires, which the new design could accommodate effectively. This adjustment not only preserved functionality, but also improved visual and structural consistency across the entire assembly.

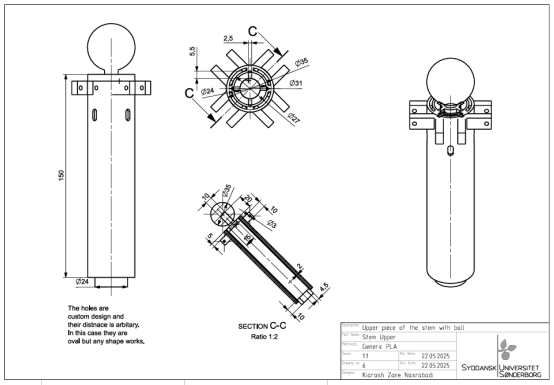


**Fig 3.4 - Upper piece of the stem with ball*

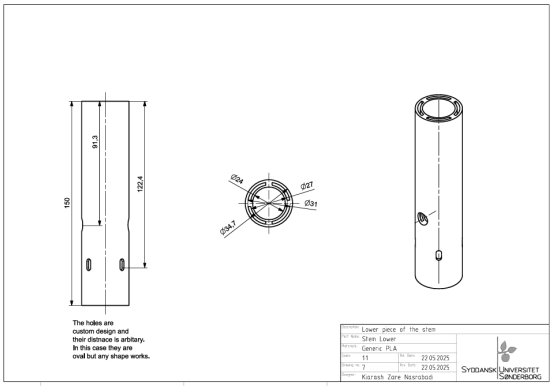


**Fig 3.5 - Lower piece of the stem*

3.2.3. Manufacturing - Laser Cutting, 3D Printing

Additional components that had to be designed included the shaft, pulley, battery holder, Arduino mount, and motor driver enclosure. These parts were relatively straightforward to

design and required minimal time for modeling, 3D printing, and testing for proper fit and tolerances. Their simplicity allowed for quick integration into the overall assembly without major adjustments.

I. Laser Cutting

All laser-cut parts, materials used, cutting settings, and the quality and outcome of each job.

Design	Material	Thickness	Number of Laser cuts	Cut Type
Floor 1	MDF	3 mm	1	Cut Only
Floor 2	MDF	3 mm	1	Cut only

**Table 3.1- Laser Cutting Outcome*

We have identified a few areas for improvement in the laser cutting process based on our laser cuts. We used the same settings - 100% power at 4 mm/s - for both floors of the design, which resulted in noticeable burn marks along the edges and the bottom side of the plate due to excessive heat. While these marks do not compromise the structural or functional integrity of the plates, they negatively impact the visual quality of the final product. To improve aesthetics without sacrificing performance, we would next increase the cutting speed to 16 mm/s and adjust the laser power to a range of 90% (minimum) to 100% (maximum). Implementing this power modulation (reducing power slightly during directional changes, especially around edges) can help prevent overburning and reduce browning. These revised settings are based on a few tests with 4 mm MDF and should translate well to the 3 mm MDF used in this project. Improving these parameters will result in cleaner cuts, and better visual appeal.

II. 3D Printing

Parts designed and 3D printed for the project, including material, print settings, print quality, and design changes. The connecting Exploded View can be found at the bottom of Appendix section 8.1. Machine and Technical Drawings.

Design	Fragile	Moving or Stationary	Color	Supports needed	Number of prints	Print time/ each
Servo motor bottom	No	Stationary	Black	Yes	2	1 Hour 30 Mins
Servo motor Top	No	Stationary	Black	Yes	2	2 Hours
Sliding top	No	Moving	Shadow Grey/Black	No	2	30 Mins
Potentiometer Gear	Yes	Moving	Terminator Grey	Yes	2	30 Mins
Solar Panel Arms	No	Stationary	Slate Grey	Yes	4	1 Hour

LDR Holders	No	Stationary	Terminator Grey	Yes	4	25 Mins
Socket	No	Moving	Dark Grey	Yes	1	1 Hour
Stem Upper	Yes	Stationary	Slate Grey	Yes	1	6 Hours
Stem Lower	No	Stationary	Slate Grey	Yes	1	5 Hours
Servomotor Holder	Yes	Stationary	Terminator Grey	Yes	1	2 Hours
Arduino	Yes	Stationary	Black	Yes	1	30 Mins
Motor Driver Case	No	Stationary	Black	No	1	1 Hour
Motor Driver Sliding Plate	No	Stationary	Black	No	1	30 Mins
Pulleys	No	Stationary	Slate Grey	No	4	15 Mins
Shafts	Yes	Stationary	Slate Grey	No	4	10 Mins
Battery Holders	No	Stationary	Terminator Grey	No		1 Hour
Pot	No	Stationary	Slate Grey	Yes	1	13 Hours

*Table 3.2 - 3D Printing Outcome

3.3. Visual Appeal Implementation

For the Nextion display, we agreed to first focus on coding and connecting it to the other components due to the limited time frame we were working with. We still had to include the buttons and other such components in the Nextion Editor, of course, to actually program the display, but we left the actual design for last in order to optimize our time management. This, along with any other visual appeal aspects, was part of our optional project requirements, as we established them in our problem formulation.

In the final week of the project, while our primary focus was soldering the final circuit and completing the assembly, we also dedicated time to improve the visual appeal of the product. Since our design is customer-oriented (with the intention of users placing it at their doorsteps and using it regularly), we prioritized user-friendliness. To support this, we developed an intuitive and visually appealing UI for the Nextion display to ensure a smooth and engaging experience for the customer.



Fig 3.5, Fig 3.6, Fig 3.7, Fig 3.8 - Nextion Display Background

While creating the final look for the display, we used two backgrounds we found on Pinterest - one for the home screen[1] and one for the other screens[2]. We also hand-drew the main icons and then put the project together in Gimp. We chose Gimp, as the ones of us working on the design were already somewhat familiar with the software, so the design process went a lot smoother. For the battery monitor and keypad, we actually created multiple backgrounds that we imported to the project in the Nextion Editor. This way, we could just switch the screen to the relevant one which was much easier to code than it would have been to program the display to change bits of the screen. Creating the buttons and other objects was easy once the pictures were imported - we just made them transparent and placed them over the button drawings in the background. At this point, the Nextion Editor code was complete, so we just had to move pre-existing objects around a bit.

4. Electrical and Software Tasks

In this section, we discuss both the electrical and programming aspects of the project, as they are closely related. Circuit assemblies will be explained alongside the piece of code needed for each sub-part's operation. As we wanted to be certain that each component and the related piece of code worked before composing the final circuit and program, we tested each of the following sub-systems separately.

4.1. Solar Panels

4.1.1. Solar Panel Testing and Behavior in Real Conditions

Our first step when beginning the work with our components was to assess how our solar panels performed in actual sunlight. This was critical for determining how much we needed to adjust the output to be compatible with the rest of the circuit, particularly the battery system. Although the panels were rated by the manufacturer, real-world conditions often differ due to factors like atmospheric diffusion, angle of incidence, and thermal effects.

To understand this better, we measured the open-circuit voltage (Voc) and short-circuit current (Isc) of each panel under direct sunlight at two different times of day - once at 11 AM

[1] "iPhone Wallpaper - Sunflowers Illustration." *Pinterest*, 25 May 2025, hu.pinterest.com/pin/840976930457870043.
[2] "Rawpixel." *rawpixel.com*, www.rawpixel.com/image/558733/sunflower-pattern?referral=103&source=pinterest.

and again at 3 PM. Both of these timings were under the peak sun hours in Sønderborg. These measurements gave us a better understanding of how panel performance varies with changing sunlight intensity. For each reading, we ensured the panels were angled toward the sun on a clear, sunny day.

The following were the results:

**Table 4.1 - Measurements 1*

Solar Panel	Voc (V)	Isc (mA)
1	6.89	76.8
2	6.91	72.4
3	6.80	70.4
4	6.82	72.7

Hence, Pmax = Voc * Isc = 0.500929125 W ≈ *0.50 W*

**Table 4.2 - Measurements 2*

Solar Panel	Voc (V)	Isc (mA)
1	6.96	70
2	6.94	70
3	6.98	72
4	6.97	71

Hence, Pmax = Voc * Isc = 0.492596875 W ≈ *0.49 W*

Interestingly, the Voc values we recorded were slightly below the manufacturer's stated values. While this might seem concerning at first, it can be explained by the inherent internal impedance of the multimeter we used; even when no load is applied, the multimeter draws a minuscule current that can slightly affect readings. Despite that, the panels still delivered about 0.5 watts each under full sun, which matched the expected performance when current and voltage were taken together.

These panels were clearly adequate for the project, as they provided enough power to charge a lithium-ion battery. In fact, their produced voltage was actually a little too high and hence needed to be lowered using additional circuitry.

4.1.2. Managing Solar Panel Output with Diodes for Compatibility

After confirming that the solar panels could deliver reliable power, we needed a way to regulate their voltage output. For this, we decided to integrate a TP4056 battery charging module. This charging module is necessary as it regulates the power output and provides safety features such as thermal overload protection, short circuit prevention, and overvoltage protection. For our application we especially need the overvoltage protection as we do not want to blow up our circuitry due to too much voltage input. The TP4056 battery charging module has a recommended input range of 4.5–5.5 volts. However, our solar panels were producing up to 6.4 volts under load, which risked damaging the module. Hence, even before letting the wires connect to the charging module, we had to lower the output voltage of the solar panels by at least 0.9V.

We considered several approaches to voltage regulation, including linear regulators and buck converters. However, these solutions added unnecessary complexity and cost for our relatively simple system. A more practical solution was to use diodes, which have a predictable forward voltage drop. For this, we tested a few diode types by measuring their voltage drops using a multimeter. The experimental results were also double checked using the corresponding datasheets found for each diode online.

The following image shows the values of the diodes we tested and their respective voltage drops:



**Fig 4.1 - Values of Diodes Tested*

The BAX18 diode emerged as the best choice, offering a forward voltage drop of around 1V. By placing one diode in series with each solar panel's positive output, we were able to bring the combined voltage down to a safe level for the TP4056. Additionally, these diodes had the added benefit of preventing reverse current flow, which could otherwise damage the solar panels or reduce efficiency.

This simple yet effective choice balanced technical reliability with cost and ease of implementation.

4.1.3. Designing the Power Delivery System

With regulated solar panel outputs feeding into the TP4056 charging module, we constructed a power flow that would ensure continuous energy delivery while protecting each component. The circuit was designed so that two solar panels in parallel would feed into each TP4056 module through a BAX18 diode. This setup allowed us to maintain voltage while doubling current output - ideal for charging.

From the charger, power flowed to two 3.7V lithium-ion batteries connected in series, resulting in a combined output of 7.4V. This voltage was suitable for powering the Arduino Nano, which served as the microcontroller for our system. To verify the setup, we monitored charging behavior using both multimeter readings and the TP4056's onboard LEDs: red for charging, blue for fully charged batteries, and off if the system was not connected or generating enough power. These visual indicators let us test if the circuit we built was actually working or not.

The final Solar panel + Diode + Battery Charging Module + Battery + Arduino are as indicated in *Figure 4.2* (component compatibilities have also been indicated).

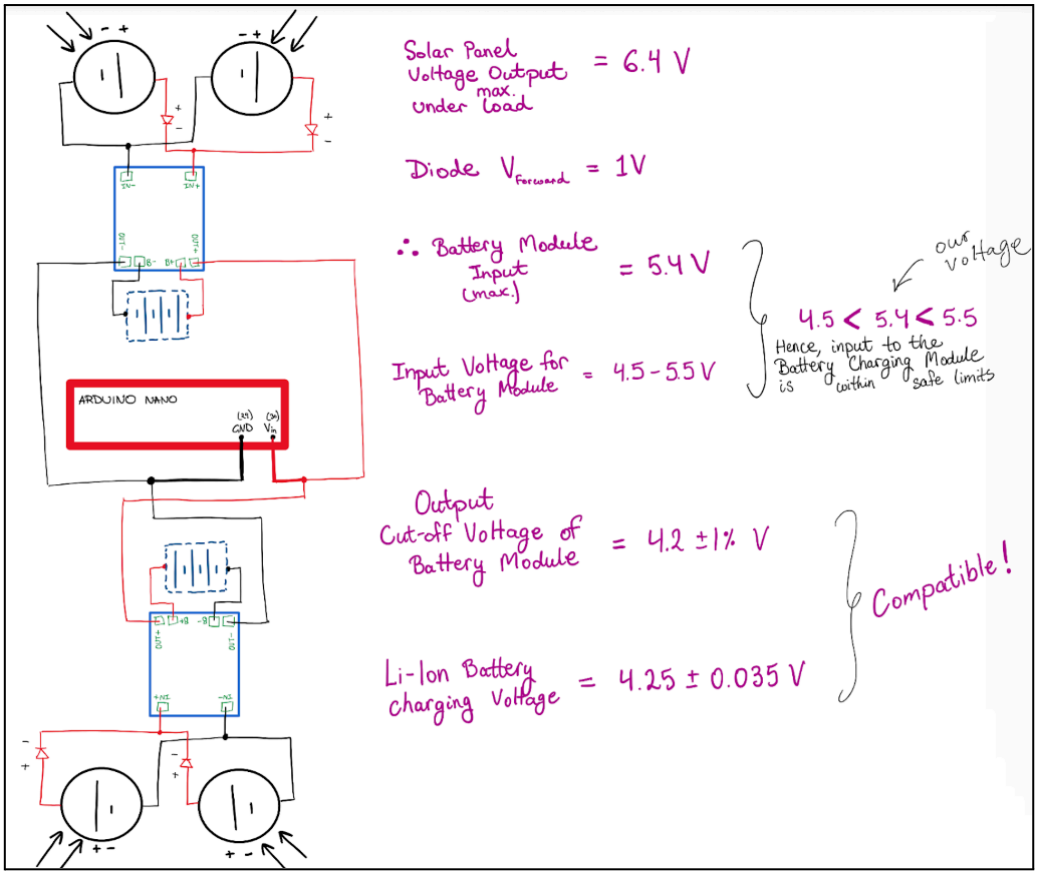


*Fig 4.2 - Power Delivery System

NOTE: The compatibilities shown in the right-side of the image are for each solar panel separately. Remember, no changes to the voltage is made even with 2 solar panels connected to the same charging module as the solar panels are connected in parallel (hence, no change to voltage).

Furthermore, in total, the batteries together will be able to supply 3.7 + 3.7V = 7.4V to the Arduino. To power the microcontroller, it is recommended for the power source to supply between 7 to 12 V - we are within this range.

4.2. Servo Motor

This section details the code for controlling servo motors, including functions for operation and energy optimization to increase efficiency.

4.2.1. PWM initialization function

This is the PWM initialisation function. This will let us control the speed of the motors.

```
84  // === PWM Init ===
85  void pwm_init(void) {
86      DDRB |= (1 << ENB) | (1 << ENA);
87      TCCR1A = (1 << WGM10) | (1 << COM1A1) | (1 << COM1B1);
88      TCCR1B |= (1 << WGM12);
89      OCR1A = 0; OCR1B = 0;
90  }
```

*Fig 4.3 - PWM initialization function

Line 86: Sets the output pins for the PWM. These correspond to the 2 motor output pins we have defined earlier. They correspond to D9 and D10 on the microcontroller.

Line 87 - 88: TCCR1A Timer/Counter1 Control Register A. WGM10 sets the waveform generation mode (along with WGM12 below) to Fast PWM, 8-bit. COM1A1 and COM1B1 set non-inverting PWM for output pins OC1A and OC1B.

17.10.1 TCCR1A – Timer/Counter1 Control Register A

Bit	7	6	5	4	3	2	1	0	
	COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10	TCCR1A
Read/Write	R/W	R/W	R/W	R/W	W	W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

*Fig 4.4 - TCCR1A Timer/Counter1 Control Register A [3]

00 and 01 (first two rows) we did not use because they do not give the PWM output. We do not need the reserves logic hence the inverting mode is not used.

Table 17-3. Compare Output Mode, Fast PWM(1)

COM1A1/COM1B1	COM1A0/COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	WGM13:0 = 15: Toggle OC1A on Compare Match, OC1B disconnected (normal port operation). For all other WGM13:0 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match, set OC1A/OC1B at BOTTOM, (non-inverting mode)
1	1	Set OC1A/OC1B on compare match, clear OC1A/OC1B at BOTTOM, (inverting mode)

*Table 4.3 Fast PWM Output Mode [3]

Line 89: Duty cycles are initially set to 0.

4.2.2. Motor A and B control setup functions

This code defines direction and speed control for two Servo motors (Motor A and Motor B) using PWM output from Timer1 (OCR1A & OCR1B). OCR1A corresponds to Motor A and OCR1B corresponds to Motor B. Earlier, during the PIN definitions, we assigned IN3 to PD5 and IN4 to PD6 for Motor A and IN1 to PD3 and IN2 to PD4 for Motor B.

[3] Microchip Technology Inc. megaAVR® Data Sheet. DS40002072A, 2018, www.microchip.com. Accessed 1 Apr. 2025.

```
94   // === Motor A Control ===
95   void motorA_set_direction(uint8_t forward) {
96       if (forward) { PORTD |= (1 << IN3); PORTD &= ~(1 << IN4); }
97       else { PORTD &= ~(1 << IN3); PORTD |= (1 << IN4); }
98   }
99   void motorA_set_speed(uint8_t speed) { OCR1A = speed; }
100  void motorA_stop(void) { OCR1A = 0; }
101
102  // === Motor B Control ===
103  void motorB_set_direction(uint8_t forward) {
104      if (forward) { PORTD |= (1 << IN1); PORTD &= ~(1 << IN2); }
105      else { PORTD &= ~(1 << IN1); PORTD |= (1 << IN2); }
106  }
107  void motorB_set_speed(uint8_t speed) { OCR1B = speed; }
108  void motorB_stop(void) { OCR1B = 0; }
```

Fig 4.4 - Motor A and B Control Setup Functions

The following is the direction control logic for Motor A and B:

IN3 (PD5) **IN1 (PD3)**	**IN4 (PD6)** **IN2 (PD4)**	**Motor Direction** **(based on our wiring)**
1	0	Forward (clockwise)
0	1	Backward (counter-clockwise)
0	0	Nothing (stop)
1	1	Nothing (stop)

Table 4.4 - Direction Control Logic for Motor A and B

Line 99 and 107: These lines set the speed of motor A and B, respectively, by adjusting the PWM duty cycle. Assigns the speed value to OCR1A and OCR1B. This sets the PWM duty cycle to correspond to speed. In practice, this is achieved because the motor driver receives PWM signals and controls motor speed accordingly.

Lines 100 and 108: These lines stop the motors A and B, respectively, by setting OCR1A and OCR1B to 0 - in order words, the duty cycle is reset to 0%.

Here, it is important to mention, however, that the OCR1A used in this function differs from Line 42 where we set OCR1A = 249. This is because the same register is being used for different contexts in the same code. Setting “OCR1A = 249;” is used for Timer1 Compare Match A in *millis()* in CTC Mode. Whereas, “OCR1A = speed;” is used for PWM duty cycle (motor speed) in PWM mode.

In our implementation, we discovered - although very late in the project period - that we were accidentally using Timer1’s Compare Match Register A (OCR1A) for two conflicting purposes: generating millisecond timestamps via CTC mode (for the *millis()* function), and

simultaneously controlling PWM output for Motor A. This oversight stemmed from the fact that the *millis()* function was only being used to print timestamps when motion was detected via the external interrupt on INT0, and not for any time-critical logic or control flow in the program.

At the time, the purpose of keeping track of time was to print the timestamps onto the Nextion display every time motion was detected to enhance product feedback and usability. Given that the *millis()* output appeared to be functional, we initially assumed the timing values were accurate. However, we now understand that Timer1 cannot operate correctly in both CTC and Fast PWM modes simultaneously, as each mode requires a different configuration of the TCCR1A/TCCR1B registers, and both attempt to use OCR1A for separate purposes.

Technically, this creates a conflict:

- *millis()* expects Timer1 to operate in CTC mode with OCR1A triggering an interrupt every 1 ms.
- PWM generation for Motor A expects Timer1 to run in Fast PWM mode, with OCR1A controlling duty cycle output on pin OC1A (PB1).

Due to this conflict, the timing values returned by *millis()* were likely corrupted or unreliable, as the timer was not consistently in the correct mode for accurate interrupt generation.Despite the severity of this error, it did not impact the functional behavior of our system because the motion detection logic does not rely on the timestamp value - it only triggers a print statement in the Motion Interrupt Handler. This means the system's motor control, LDR averaging, EEPROM updates, and sensor logic were unaffected. Hencewhy, we had overlooked this issue for so long.

For a better and correct implementation, it would be best to dedicate separate timers to these tasks - using, for example, Timer0 or Timer2 for *millis()*, while reserving Timer1 solely for PWM generation. This would avoid register conflicts and ensure accurate, deterministic timing behavior across all features. This would mean changing the registers that would be suitable for the new Timer - this will not be a challenge as we just have to find the appropriate registers for another Timer in the Atmega32A Datasheet and change those to correspond to the new Timer.

4.2.3. Rotate motor function

This function rotates a motor (A or B - not yet specified) toward a specified angular target using feedback from the connected potentiometer (for angle) and logs LDR values for light comparison. It runs until the motor reaches a desired target angle with ±1° tolerance (Line 128).

```
122  // === Rotate Motor Function ===
123  void rotate_motor(const char* motor_name, const char* pot_name,
124                    float target, float* current, uint8_t pot_pin,
125                    void (*set_dir)(uint8_t), void (*set_speed)(uint8_t), void (*stop)(),
126                    float (*ldr_avg)()) {
127      printf("[INFO] Rotating Motor %s to %.2f° using Pot %s\r\n", motor_name, target, pot_name);
128      while ((*current < target - 1.0f) || (*current > target + 1.0f)) {
129          if (*current < target) {
130              printf("→ Motor %s: FORWARD\r\n", motor_name);
131              set_dir(1);
132          } else {
133              printf("← Motor %s: REVERSE\r\n", motor_name);
134              set_dir(0);
135          }
136          set_speed(255);
137          _delay_ms(200);
138          *current = (analog_read(pot_pin) * 300.0f) / 1023.0f;
139          printf("   ↳ Pot %s angle: %.2f°, LDR Avg: %.2f\r\n", pot_name, *current, ldr_avg());
140      }
141      stop();
142      printf("[DONE] Motor %s reached %.2f°\r\n", motor_name, *current);
143  }
```

Fig 4.5 - Rotate Motor Function

The actions performed by the motor (forward or reverse) is printed on the serial monitor so we know what the code is doing right now. This was especially useful during the testing stage where this approach made debugging such as whether the wire connections were correct were much easier.

Line 136: The voltage supplied by the batteries is max 7.4 V. The voltage needed to keep the servo motors moving individually is minimum 6.5V. And even higher voltage, about 7V, is the threshold voltage needed to initially move the servomotors. In addition, due to the added friction from the additional gears, even more voltage is required to initially move the motor + gear assembly. For this reason, we decided to keep the PWM speed at its max (255). In our code, we first move Motor A into position, followed by Motor B at some set interval. This decision was made because there is not enough voltage available to run both motors simultaneously.

The *rotate_motor()* function leverages pointers to create a function that can be called for both functions. String pointers (*const char* motor_name*, *const char* pot_name*) are used to dynamically label output messages, ensuring *printf()* statements accurately indicate which motor and potentiometer are being referenced. The float pointer *float** current is essential to track the angular position of a motor. By dereferencing this pointer, or in other words accessing its address, the function updates the motor's current angle based on real-time readings from the corresponding potentiometer.

The analog value read from the ADC (ranging from 0 to 1023) is converted into a physical angle in degrees (0° to 300°) based on the known operating range of the 10k potentiometer we are using. Because current is passed by reference, the updated angle persists outside the function scope, allowing the main loop to make further decisions based on the new motor state.

In addition, the function also uses function pointers *(void (*set_dir)(uint8_t)*, *void (*set_speed)(uint8_t)*, *void (*stop)())* to abstract the control logic of each motor. These function pointers are set to point to the specific motor control routines (e.g., *motorA_set_direction, motorB_set_speed*, etc.) at the time of function call. This allows the same *rotate_motor()* function to execute different behavior depending on which motor is being controlled - all without duplicating code or hardcoding logic. The *ldr_avg()* pointer is similarly used to allow the function to work with light sensor data (LDRs) associated with each motor, maintaining separation of sensor feedback per motor.

4.3. Battery and Power Supply

4.3.1. Batteries Test Circuit for Nextion Display

To measure the voltage of two 3.7 V batteries, we created two separate voltage divider circuits (one for each battery). This allowed us to connect each battery's output to its own analog input on the Arduino: A6 for Battery 1, and A7 for Battery 2.

For each battery, we connected the positive terminal to the top of a voltage divider made from two resistors, and the negative terminal to GND. The midpoint between the two resistors was connected to an analog input pin on the Arduino.

We initially tried 2 10 Ohm resistors for just one battery, but the battery Voltage read was too low (0.23 V) so we figured out we needed higher value resistors. We tried different till we reached 110 kΩ (2 100 kΩ would have been also fine, but they were finished when we looked for them in the Lab). In the end used two 110 kΩ resistors per divider, which worked well for reducing current consumption while safely scaling down the battery voltage.

For each divider:

- We connected one 110 kΩ resistor between the battery's positive terminal and the analog input pin.
- Then we connected another 110 kΩ resistor between the analog input pin and GND.

This formed a simple voltage divider that reduced the battery voltage by half before sending it to the Arduino's analog pin.

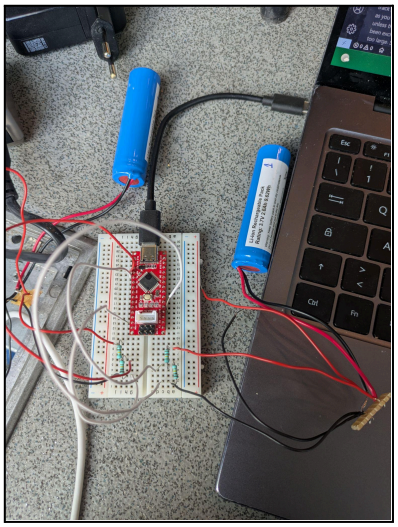


*Fig 4.6 - Batteries Test Circuit

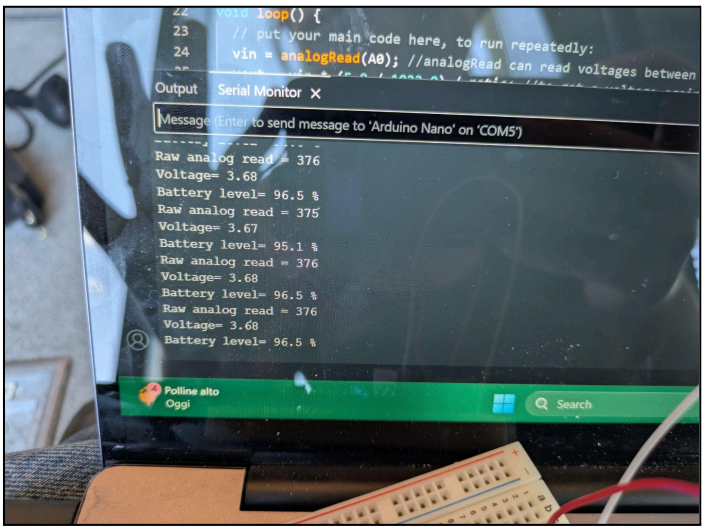


*Fig 4.7 - Serial Monitor Output

By reading the voltages on A6 and A7 (thanks to the function *analogRead* explained in page 25), we could monitor each battery individually and calculate their actual voltages. This setup helped us have each battery voltage to check if there was a problem in any of the circuits that we built for each of them or if one would have discharged more rapidly than the other (and would have shown us that there is a malfunction that we could have easily repaired by buying another battery).

Battery Voltage (V)	Voltage at Analog Pin (V)
6.0 (low)	2.4
7.4 (nominal)	2.96
8.4 (full charged)	3.36

**Table 4.5 - Battery voltage*

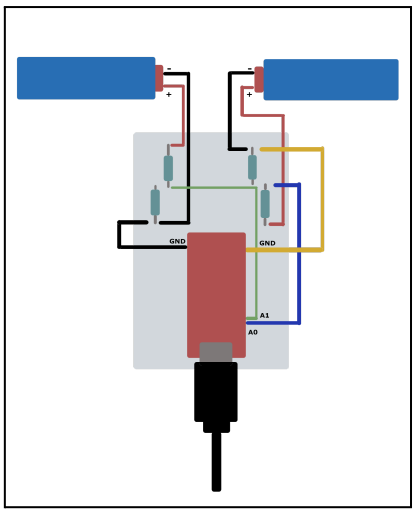
4.3.2. Readjustment of the Batteries Circuit

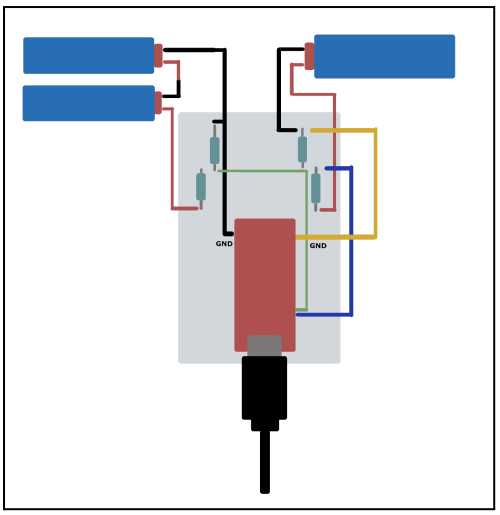
We then realized that the Voltage provided by 2 batteries was too low to make everything work (motors, Nextion display, Motion Sensor and Arduino). Furthermore, since we did not have any more pins to add another battery we thought to connect 2 batteries in series to one Pin and leave the second as it was so that we could have a final Voltage of at least 9V. We had to do it by making sure the Arduino could safely read the voltages without damage (for an Arduino the safe range is from 0V to 5V, but we should never reach the limits).

Here is our second battery circuit:

- We connected 2 3.7 V batteries[4] in series by linking the positive terminal of the first battery to the negative terminal of the second battery: in this way, the maximum Voltage will be 7.4 V (just nominal Voltage for when it is charging).
- For the voltage divider on the series batteries, we used a 110 kΩ resistor connected from the battery red wire (positive) to the Arduino analog input pin. Then, we connected a 150 kΩ resistor from the Arduino analog input pin down to ground (GND). Thanks to the resistors the 7.4 V would go down to around 4.2 V, which is safe for the Arduino's analog input.
- For the single battery connected to the other analog pin (A7), we used 2 110 kΩ resistors as a voltage divider. One resistor goes from the battery positive wire to the analog pin, and the other from the analog pin to the GND. This halves the voltage, making it safe to measure.

[4] Battery University. "BU-302: Series and Parallel Battery Configurations." *Battery University*, 3 Mar. 2022, batteryuniversity.com/article/bu-302-series-and-parallel-battery-configurations.





** Fig 4.8 - Diagram Battery circuit 1", 2 batteries each in a circuit with 2 110kOhm resistors*
** Fig 4.9 - Diagram Battery circuit 2", 3 batteries: one connected as the batteries in Diagram 1 and 2 connected in series in a circuit with 2 resistors(one 110kOhm and one 150kOhm)*

4.4. Motion Sensor

4.4.1. Motion Sensor Circuit

As we were not familiar with motion sensors we had to do some research on how to set up the circuit. Luckily, we found a very nice tutorial on how to test it and set it up.

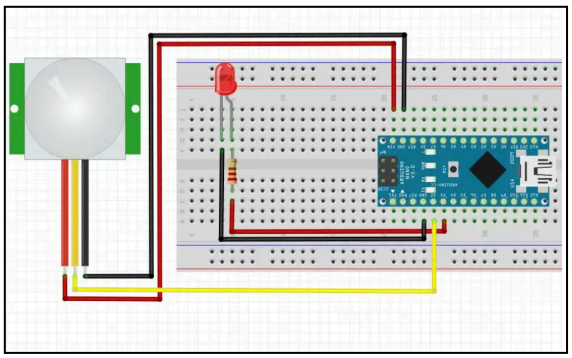


**Fig 4.10 - Motion Sensor Circuit Test*

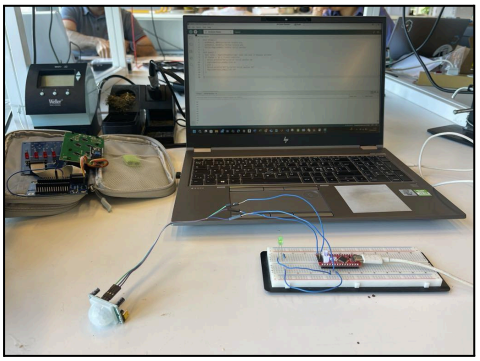


**Fig 4.11 - Serial monitor Output*

At first, we tried testing the motion sensor using the circuit model and code from a hobby electronics website called SriTu Hobby, but we had chosen the wrong circuit setup and had no success, as the only response we got was "no motion detected". It is worth noting that we also included an LED in our circuit, so as long as there is motion detected, the LED turns ON. This way, it would have been easy to see whether the code was correct or not. With this setup and code we got no response from the LED, however. Instead, the LED on the Arduino was turning ON and OFF, seemingly at random, making no sense.

After many different approaches, we changed both the circuit and the code. Once we had a good result and a working motion sensor, we had to add it to the main circuit with the rest of the components. As we initially tested the code using Arduino IDE, but agreed to use Visual Studio Code and C language for the final code (see 4.4.2), we had to convert our code before finally including it in the combined code. What this code basically does is that for as long as there is motion detected the LED turns ON.

```arduino
void setup() {
  pinMode(2, INPUT);//define arduino pin
  pinMode(3, OUTPUT);//define arduino pin
  Serial.begin(9600);//enable serial monitor
}
void loop() {
  bool value = digitalRead(2);//get value and save it boolean veriable
  if (value == 1) { //check condition
    Serial.println("ON");//print serial monitor ON
    digitalWrite(3,HIGH);//LED on
  } else {
    Serial.println("OFF");//print serial monitor OFF
    digitalWrite(3,LOW);//LED off
  }
}
```

*Fig 4.12 - Motion Sensor Code *Arduino IDE**

4.4.2. Motion Sensor Code

I. Motion detection function

This is the code for the Motion detection. We are using interrupts because it will allow the microcontroller to immediately respond to changes on the motion sensor pin (INT0). This will spare us from always constable checking the INT0 pin constranty in the main loop. Using interrupts also allows the code to quickly react even if quick or brief signals are observed - which would not have been the case otherwise.

```arduino
// === Motion Interrupt Setup ===
void motion_interrupt_init(void) {
    EICRA |= (1 << ISC00); // Interrupt on any logical change on INT0 (PD2)
    EIMSK |= (1 << INT0);  // Enable INT0 interrupt
    DDRD &= ~(1 << MOTION_PIN); // Set MOTION_PIN as input
    PORTD &= ~(1 << MOTION_PIN); // Disable pull-up (assuming external pull-down or sensor handles it)
}
```

*Fig 4.13 - Motion Detection Function *C**

Line 61: EICRA (External Interrupt Control Register A) configures how the external interrupts INT0 and INT1 are triggered. This sets the ISC00 bit in EICRA, configuring external interrupt 0 (INT0) to trigger on any logical change (both rising and falling edges) on the INT0 pin (which corresponds to pin PD2). The ISC01 bit remains cleared, so the interrupt triggers on any change rather than only rising or falling edge.

Table 14-2. Interrupt 0 Sense Control

ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

Table 4.6 - Interrupt sense Control [3]

Line 62: EIMSK (External Interrupt Mask Register) enables or disables the external interrupts INT0 and INT1. This line specifically enables the external interrupt on the INT0 pin.

INT0 bit (bit 0) enables the external interrupt 0 when set to 1. When the bit is cleared, the interrupt is disabled and then remains to the main code.

Line 63: Sets MOTION_PIN, earlier defined to PD2, as input. Setting the bit 1 means setting the pin as output, but we are negating it, hence setting the bit 0 which is configuring PD2 as input.

II. Motion detection interrupt

This defines an ISR for the external interrupt 0 (INT0_vect), which triggers whenever there is a change on the external interrupt pin associated with INT0. Remember, we earlier enabled the interrupt triggered by the motion sensor to the external interrupt on the INT0 pin. Inside the ISR, it reads the current state of a motion sensor input pin - i.e. PORTD. If the bit corresponding to MOTION_PIN is 1 (HIGH), it prints "MOTION: DETECTED" along with the current system uptime in milliseconds, obtained by calling *millis().* Otherwise, if the pin is 0 (LOW), it prints "MOTION: NONE" with the timestamp.

For improved efficiency and to save space in the serial monitor, lines 71–72 can be omitted in future versions of the code, resulting in no output when no motion is detected. However, these lines were intentionally included during the testing phase to verify proper sensor connectivity and to confirm that the interrupt handler is triggered on every sensor state change.

```
67  // === Motion Interrupt Handler ===
68  ISR(INT0_vect) {
69      if (PIND & (1 << MOTION_PIN)) {
70          printf("MOTION: DETECTED at %lu ms\r\n", millis());
71      } else {
72          printf("MOTION: NONE at %lu ms\r\n", millis());
73      }
74  }
```

**Fig 4.14 - Motion Detection Interrupt*

4.5. Nextion Display

To improve our users' experience we designed a set of simple pages for the Nextion Display, with most intuitive symbols and clean layout so that they can easily communicate with the system (e.g. changing the sound for the speaker) and to understand the information from the screen without an extensive technical knowledge.

We wanted to show the following information from the Arduino communication:

1. Battery level (and Voltage), so we can see from outside if there is any problem with them (e.g. if they are not charging anymore and battery level is 0 there is probably a problem with connection between the Arduino and the Solar Panels);
2. Number of times and times of the day when the Motion Sensor detects something (in 48 hours so the user does not have to delete them manually every day).

We also decided to create a Pin Pad that allows us to go to the Nextion settings and information so only the intended user can access them.

Below, we will discuss the functions used to allow the communication between the Arduino and the Nextion Display and how the data is shown on the screen.

incrementClock() function

We added one second to the time held in our custom Clock structure each time the *incrementClock()* function is used. We increased the minutes and reset the seconds to zero when they reached 60. In a similar manner, we increased the hours and reset the minutes when they reached 60. At last, we reset them to zero when the hours hit 24. This made it possible for us to keep track of hours, minutes, and seconds on a basic 24-hour clock.

```
45  // ===== CLOCK FUNCTIONS =====
46
47  void incrementClock(Clock* clk) {
48      clk->seconds++; // Increment seconds
49      if (clk->seconds >= 60) {
50          clk->seconds = 0;
51          clk->minutes++; // Increment minutes
52          if (clk->minutes >= 60) {
53              clk->minutes = 0;
54              clk->hours++; // Increment hours
55          }
56          if (clk->hours >= 24) {
57              clk->hours = 0; // Reset after 24 hours
58          }
59      }
60  }
```

Fig 4.15 - incrementClock() Function

The *getSeconds()* method

The total seconds that have passed since midnight (or any desired time of beginning the code) based on the current clock time was determined by the *getSeconds()* function. We accomplished this by adding the seconds and multiplying the hours by 3600 and the minutes by 60. For the purpose of tracking time-based events and comparing timestamps, this provided us with a simple single integer that represented the time.

```
62  uint32_t getSeconds(const Clock* clk) {
63      return clk->hours * 3600UL + clk->minutes * 60UL + clk->seconds;
64  } // Convert time to total seconds
65
```

Fig 4.16 - The getSeconds() Method

analogRead()[5]

We developed a general *analogRead()* function that returned the ADC reading from an analog channel after receiving the channel number as input. After adjusting the ADMUX register to choose the desired analog input channel and setting the ADSC bit to initiate the conversion, we waited for it to complete. We then gave back the 10-bit ADC value. We also made two helper functions, *analogRead_A6()* and *analogRead_A7()*, which simply called *analogRead()* with the appropriate channel numbers for pins A6 and A7.

```
67      int analogRead(uint8_t channel)
68  int analogRead(uint8_t channel) {
69      ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);  // Select ADC channel
70      ADCSRA |= (1 << ADSC); // Start conversion
71      while (ADCSRA & (1 << ADSC)); // Wait for conversion to finish
72      return ADC;   // Return result
73  }
74
75  int analogRead_A6() { return analogRead(A6); }  // Read from A6 (ADC6)
76  int analogRead_A7() { return analogRead(A7); }  // Read from A7 (ADC7)
77
```

Fig 4.17 - analogRead() Function [3]

[5] Developing in C for the ATmega328P: Function - analogRead() | Wellys Dev. 12 Sept. 2022, wellys.com/posts/avr_c_function_analogread.

sendToNextion()

We used UART to deliver a string instruction to the Nextion display in *sendToNextion()*. In order to accomplish this, we sent the string's characters one at a time, followed by three 0xFF bytes, which the Nextion display needs to indicate the conclusion of a command. This made it possible for us to use the microcontroller to update text fields and other user interface components on the screen.

```
80  void sendToNextion(const char* cmd) {
81      for (size_t i = 0; cmd[i] != '\0'; i++) {
82          putchar(cmd[i]);  // Send command one character at a time
83      }
84      putchar(0xFF);  // Termination bytes required by Nextion
85      putchar(0xFF);
86      putchar(0xFF);
87  }
```

*Fig 4.18 - sendToNextion() Function

function constrain()

To limit a floating-point number to fall between two boundaries (a minimum and a maximum), we built a straightforward *constrain()* function. We returned the value unmodified if the input was below the minimum, the maximum if it was above the maximum, and the minimum otherwise. This helped to maintain battery percentage readings between 0% and 100%.

```
91  float constrain(float x, float a, float b) {
92      if (x < a) return a;
93      if (x > b) return b;
94      return x;
95  }
96
```

```
avg_perc = constrain(avg_perc, 0, 100); // Clamp between 0-100%
```

*Fig 4.19 - Function constrain()

***cleanupOldEvents()* function**

To avoid filling up the event list forever, we periodically cleaned up events older than 24 hours. We looped through all events, and for any active event whose timestamp was more than 24 hours older than the current time, we marked it inactive and cleared the corresponding display entry on the Nextion screen.

```
116
117 void cleanupOldEvents(uint32_t nowSeconds) {
118     for (uint8_t i = 0; i < MAX_EVENTS; i++) {
119         if (events[i].active && (nowSeconds - events[i].created_at_seconds) >= SECONDS_IN_24H) {
120             events[i].active = 0; // Deactivate event older than 24h
121
122             char cmd[30];
123             snprintf(cmd, sizeof(cmd),"Clock.t0.txt=\"\"", i + 1);
124             sendToNextion(cmd);// Clear event from screen
125         }
126     }
127 }
```

```
cleanupOldEvents(nowSeconds); // Remove expired events
```

*Fig 4.20 - cleanupOldEvents() Function

***main()* function overview**

In the *main()* function, we first initialized the UART communication and the ADC. We created a Clock starting at midnight, but that can be changed from any hour based on when we start the program.

Then, inside an infinite loop the code:

- reads analog voltages from pins A6 and A7 (the two battery voltage inputs);
- converts the raw ADC values into voltages, corrected for the voltage divider ratio;
- creates the average of the two battery voltages and converts that to a battery percentage, constraining it between 0 and 100%;
- sends battery percentage and voltage data to the Nextion display to update the battery indicators;
- increments the clock by one second and updates the time display.
- checks if motion was newly detected and, if so, recorded the event and updated the display;
- cleans up any motion events older than 24 hours.

As a result we have a simple real-time system that monitors battery voltage and motion sensor events, logs recent motion events, and keeps a clock (all displayed live on the Nextion display).

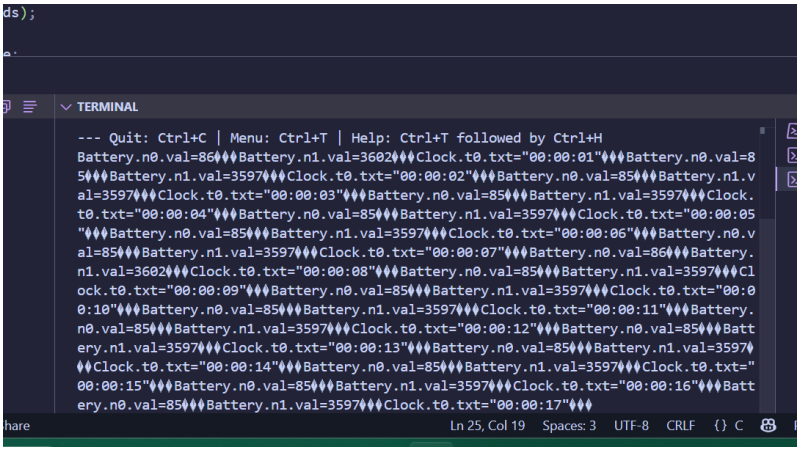


Fig 4.21 - Main Function Overview

Here, in the Terminal of VS code we can see the values send to the Nextion display:

1. Battery.n0.val=86 (Battery is the name of the page on the Nextion and n0 is the text box where the value is shown and 86 is the average battery level);
2. Battery.n1.val=3602 (same page , but the value is the average Voltage in mV);
3. Clock.t0.txt=”00:00:03” (Clock is another page and shows the time passing at every second).

During this Debug simulation the batteries were not connected to the Arduino so the values were changing randomly every second.

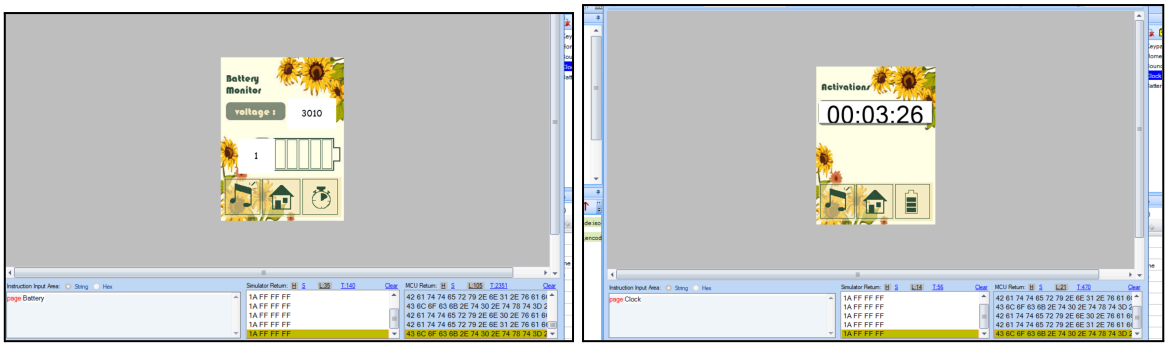


Fig 4.22 - Values Sent to the Nextion Display

Something to improve in this project is, for sure, the clock shown on the display and a better option would have been a RTC Module.

An RTC module (Real-Time Clock) is a small chip that keeps track of the current time and date even when your main device is powered off. It has its own battery, so it can keep running independently and accurately for years. It is essentially a tiny, low-power clock built into the circuit.

While using a RTC module would have provided more accurate and reliable timekeeping, especially over long periods or during power loss, our software-based clock was sufficient for this simple demonstration. For a real-world application, though, an RTC would definitely be the better choice.

4.6. Speakers

As our idea for the project involved an alarm system, it was crucial to the final execution that we can play audio using the Arduino and a speaker. There are a couple of ways to achieve this, the most commonly used one being copying the audio file to an SD card and inserting it into a connected SD card reader. However, we did not have a reader as our goal was to include this functionality in the code.

The easiest way we found for playing audio on the Arduino is by using the custom PCM library and an encoder software[6]. The idea is that we convert any audio file we would like to use to 16 bit and 8 kHz - these values make for a good quality audio, well within the human hearing range. For this, we used an online converter, then we encoded this file with the provided encoder software[7] which automatically copies it into the clipboard from where we can insert it into our code. One problem we encountered with this method though, is that even a 10 seconds long .mp3 file occupied way too many bytes causing the code to not compile. Our solution was to use a 5 seconds mp3 file instead and then play the file a couple of times using a simple loop. This was the method we used for playing the barking sound we picked for our alarm.

[6] Just Baselmans. “Make Arduino Play ANY Audio File (Without SD Card) QUICK TUTORIAL.” *YouTube*, 30 Apr. 2021, www.youtube.com/watch?v=m1HEwgHSBrs.
[7] Shveytank. “GitHub - Shveytank/Arduino-MP3.” *GitHub*, github.com/shveytank/Arduino-MP3/tree/master.

Originally, the barking would have been the only alarm sound in the project, however, while we were looking into different methods for coding the playback, we found another really neat way of doing it. This is based on the custom pitches library and arrays. Musical notes make up one array and their intended duration times make up another one. These are then called in the tone function that plays back the entire melody as part of a for loop.[8]

We found resources[9] for a wide variety of popular melodies that were already coded, so we picked 2 additional tracks from there and decided to incorporate it in the project. It is worth noting that these two melodies were not of the same array size, which we had to account for in the final code - we cut the longer melody so it fitted the size of the shorter array. This way, it was sufficient to write a single function - which will be discussed later - that could play back either of these two tracks and we only had to write a separate piece of code that dealt with the first method of playback. This latter was necessary, as we would not have been able to break down the intended barking into musical notes and play that back. The functions and the way they work will be discussed later on.

4.6.1. Playback Testing and Speaker Circuit

Generally, we tested every block of code that made up a specific functionality on its own before adding it to the final code, so it was no different for the playback codes either. The test circuit involved the Arduino, the speakers, and a transistor - this last was necessary to enhance the volume of the speaker which was originally far from an alarm volume.

When building the circuit, we based ours on a guide[10] we found. According to this, the following connections had to be made:

- pin D11 on Arduino goes to the base of the transistor,
- GND on Arduino goes to the emitter of the transistor,
- the negative end of the speaker goes to the collector of the transistor,
- and the positive end of the speaker goes to the 5V power supply on the Arduino.

The issue with this was that it was just simply not working. We knew that the problem could not be the speaker - we tested that on its own when it was directly connected to the power supply and it played the default tone programmed into it. This left us with three possible causes behind the problem:

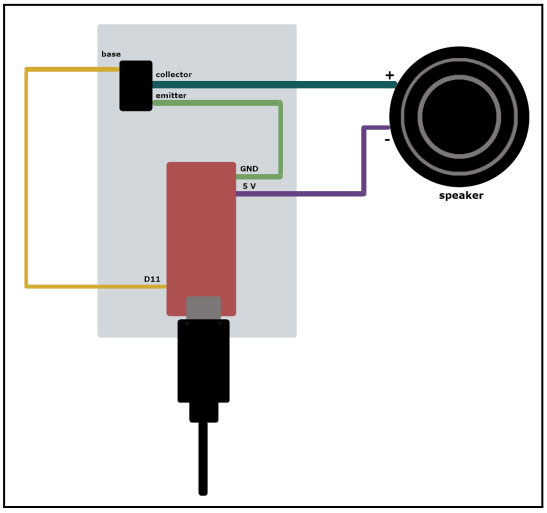
- errors in the code,
- incorrect/incompatible audio format used, or
- some issue with how the circuit was built.

The code was easy to check for any errors as most of it at this point was just copy pasted from the original. For good measure, though, we also ran a simple code for playing a single tone with only the speaker connected directly to the Arduino. This too worked, and knowing that the notes were also directly copied with the original code, one possible problem source remained: the circuit.

---

[8] “Playing Popular Songs With Arduino and a Buzzer.” *Arduino Project Hub*, projecthub.arduino.cc/tmekinyan/playing-popular-songs-with-arduino-and-a-buzzer-546f4a.
[9] Hibit-Dev. “Buzzer/Src at Master · Hibit-dev/Buzzer.” *GitHub*, github.com/hibit-dev/buzzer/tree/master/src.
[10] electronic GURU. “Talking Arduino | Playing MP3 Audio With Arduino | Arduino PCM Audio Without Audio or Mp3 Module.” *YouTube*, 3 May 2018, www.youtube.com/watch?v=F28Znry0qcw.

Now for the circuit, it is worth mentioning that we were - as with many other parts of the project - figuring it out as we went. Meaning that we looked up similar projects on forums and watched a couple of youtube tutorials, took note of the relevant parts and then built our circuit according to the description and circuit diagram above. After an entire day of testing and adjusting this circuit (adding resistors, changing the used transistor, using different pins, etc.), it turned out that the problem was the tutorial itself we were following. As we realized later, after watching another video[11] which was, by the way, based on the one we originally followed, the one we watched had the negative and positive ends of the speaker connected the opposite way as they were supposed to be. Once we switched them, the speakers were finally playing the pre-written melodies.



Audio	
Bit rate	128kbps
Channels	1 (mono)
Audio sample rate	8.000 kHz
Audio sample size	16 bit

*Fig 4.23 - Speaker Testing Circuit and MP3 File Specifications

For the barking, we had to convert the mp3 file to have a frequency of 8 kHz and a size of 16 bits. As mentioned before, the original 10 seconds of duration also had to be cut to 5 seconds due to the limited number of bytes the code could work with.

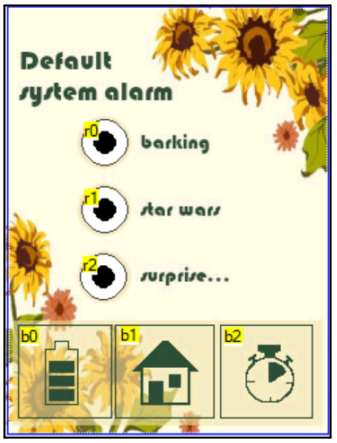


*Fig 4.24 - Poster of S.Q.B.

4.6.2. Nextion Editor Code for Alarm Sound Screen

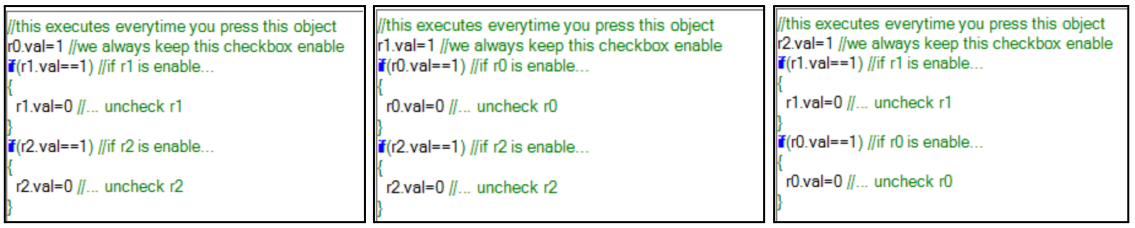
The relevant coding in the Nextion Editor concerns only the sound screen, and more specifically the three radio buttons.

What we wanted to achieve was a menu with 3 checkboxes out of which only one would be checked at any given time, that one being the selected system alarm. We decided to go with the radio checkboxes simply because we preferred their look over the simple checkbox button. For each of the buttons the same code was included in the Nextion editor with only the names of objects modified to suit the selected checkbox.

What these codes do is that they make it possible for only one of the radio checkboxes (named r0, r1 and r2 respectively) to be checked at a time. If, for example, r0 is pressed, its

[11] Electronics BD. “How to Play Audio With Arduino.” *YouTube*, 4 Oct. 2019, www.youtube.com/watch?v=aaqaAXlZbuc.

value becomes 1, while the value of r1 and r2 is automatically set to 0. These values stay this way, unless another button is pressed. Later on, we will read the values of these radio checkboxes in the main code to determine which sound was selected as default and execute the playback code based on them.



```
//this executes everytime you press this object
r0.val=1 //we always keep this checkbox enable
if(r1.val==1) //if r1 is enable...
{
 r1.val=0 //... uncheck r1
}
if(r2.val==1) //if r2 is enable...
{
 r2.val=0 //... uncheck r2
}
```

```
//this executes everytime you press this object
r1.val=1 //we always keep this checkbox enable
if(r0.val==1) //if r0 is enable...
{
 r0.val=0 //... uncheck r0
}
if(r2.val==1) //if r2 is enable...
{
 r2.val=0 //... uncheck r2
}
```

```
//this executes everytime you press this object
r2.val=1 //we always keep this checkbox enable
if(r1.val==1) //if r1 is enable...
{
 r1.val=0 //... uncheck r1
}
if(r0.val==1) //if r0 is enable...
{
 r0.val=0 //... uncheck r0
}
```

**Fig 4.25 - Nextion Editor Code for Alarm Sound Screen*

4.6.3. C Code for Choosing and Playing Default System Alarm

As previously mentioned, we agreed to separately test each sub-system of the final code and circuit. Therefore this section only discusses the setup and coding of the circuit needed for testing the default system alarm functionalities.

It is also worth noting that while the rest of the code included in this report is already written in or was converted later to C, the code in this section references the test code which was executed using Arduino IDE instead of Visual Studio Code. The reason behind this was that working in Arduino IDE we did not have to worry much about the libraries needed for the Nextion communication and we planned on handling those once we made sure that the code was correct and converted into C. However, during the testing and circuit assembly, both of our speakers broke and the ones we ordered as replacement had not arrived by the deadline of the report hand-in. This way, we could not test this code with the hardware and we did not want to potentially corrupt the already functioning parts of our final code.

The test system works as follows: On a sound selection screen the user would pick one of 3 possible alarm options. This selection is stored in the system memory. When the motion sensor is triggered, the previously selected sound is played.

This, at first, sounds quite simple and straightforward to code. However, we were using 2 different methods for playing melodies and we also had to find a way to send a signal from the display to the Arduino so that it would execute a task - namely, remember which alarm sound was chosen and then play it at the right time. The latter, we could not do in the Nextion Editor as we could for simple button presses, as the playback code would be in the Arduino code and we have no way of connecting those two. Therefore, we looked up some guides online and used those as a starting point.

I. Setup of the Nextion to Arduino Communication

According to the guide[12] we followed, in order to enable Nextion to Arduino communication, we have to declare the objects we want to use for this purpose before the main loop of the code. In our case, it would be the 3 radio buttons.

[12] InterlinkKnight. “Nextion+Arduino Tutorial #2 Sending Data to Arduino.” *YouTube*, 20 Sept. 2017, www.youtube.com/watch?v=mdkUBB60HoI.

```
//NEXTION related setup
//declare objects we are reading from the display:
//format: function name = function(page number, ID, "name") --- these are already changed to final pg.no. and IDs!!!
NexRadio r0 = NexRadio(2, 5, "r0");
NexRadio r1 = NexRadio(2, 6, "r1");
NexRadio r2 = NexRadio(2, 7, "r2");
```

Fig 4.26 - Setup of the Nextion to Arduino Communication

Next up, we create a touch event list, composed of the radio checkboxes. This would then be used in the main loop to check if any of the previously defined touch events are happening and then execute the corresponding instructions.

```
//declare touch event objects to the touch event list:
//format: &<name>
NexTouch *nex_listen_list[] =
{
  &r0, //radio checkbox added
  &r1,
  &r2,
  NULL //string terminated
};
```

Fig 4.27 - Radio checkboxes

To define the corresponding instructions to each touch event, we have to separately define them. However, as this is still outside of the main loop, we cannot run any actual pieces of code here. For this reason, we just defined some variables that would be assigned the values of each radio button (r0 to r2) by use of the ‘get’ function. These values along with the code and explanation for when and how they change were included in *Chapter 4.6.2*. Later, these variables will be used in the main loop to determine which sound was selected as default.

```
//touch events:
//each touch event will trigger one of the following sections

void r0PushCallback(void *ptr) //press event for radio checkbox r0
{
  //code for setting the default sound to WOOF WOOF
  int value_r0 = Serial.print("get r0.val"); //reads the value of r0 --- could be 0(not pressed) or 1 (pressed)

  Serial.write(0xff);
  Serial.write(0xff);
  Serial.write(0xff);
}

void r1PushCallback(void *ptr) //press event for radio checkbox r1
{
  //code for setting the default sound to STAR WARS
  int value_r1 = Serial.print("get r1.val");

  Serial.write(0xff);
  Serial.write(0xff);
  Serial.write(0xff);
}

void r2PushCallback(void *ptr) //press event for radio checkbox r2
{
  //code for setting the default sound to SURPRISE
  int value_r2 = Serial.print("get r2.val");

  Serial.write(0xff);
  Serial.write(0xff);
  Serial.write(0xff);
}
```

Fig 4.28.1 - Touch Event Callback Functions

In the setup section, we establish the communication with the display at Baud rate 9600 as we always do. What we have to do in addition to this, is to register the event callback functions to each touch event and set our previously selected buzzer pin to audio output mode. Note that there is a button press enable - the relevance of this will be discussed in the next section.

```
void setup() {

  Serial.begin(9600);

  //register the event callback functions of each touch event:
  r0.attachPush(r0PushCallback); //radio checkbox press
  r1.attachPush(r1PushCallback);
  r2.attachPush(r2PushCallback);

  pinMode(buzzer_pin, OUTPUT); //setting our previously defined D11 as audio output pin
  pinMode(buttonPin, INPUT_PULLUP); //enable internal pull-up resistor for button --- only needed for testing

}
```

Fig 4.28.2 - Setup Function: Initializing Serial Communication and Registering Touch Events

II. Main Loop Breakdown

We call the touch event list in the main loop which checks if any of the registered touch events (for r0 to r2) has recently happened and executes the code that was written as corresponding instructions. In this case, it would be getting the values of r0, r1 and r2. Then in the next section we define the variable default_alarm. This we need to operate our first function - which will be discussed later on in the Functions section - with more ease.

The value of default_alarm changes depending on which sound is selected as the default system alarm audio. We check the values of r0, r1 and r2 in main and assign values to default_alarm based on that. If either r1 or r2 is selected, we use the playback method with arrays and the pitches library. If r0 is selected, we use the playback method with the PCM library and the encoded audio.

```
void loop() {

  //checking the list of touch events we defined to know if any of them just happened
  //and then execute the corresponding instructions
  nexLoop(nex_listen_list); //in this case, the corresponding instruction would be assigning a value to default_alarm


  if(value_r0 == 0){ //if it's NOT r0 selected

    if(value_r1 == 1){ //it can be r1 selected if r1=1
      default_alarm = 1;
    }
    else{ //the remaining one case is if r2 is selected
      default_alarm = 2;
    }

    //here goes function no. 1 --- if r1 or r2 was pressed, then the melody[] and durations[] get populated
    void melodyFunction (default_alarm, melody_sw, durations_sw, melody_spr, durations_spr, melody, durations, sizeof(durations)/sizeof(int));

  }

  //as the barking is based on a different playback type, this should be enough for now
  else{ //when r0 is selected
    default_alarm = 0;
  }
```

Fig 4.29 - Alarm Sound Based on User Input

For testing purposes, we used a simple button press instead of setting up the playback with the motion sensors right away. Our thought process being, if something was wrong, debugging would be easier with only one of them to handle at a time. Once we made sure they were working separately, we combined and tested them together.

```
//inside a button press loop (later: motion sensor triggered loop)

for(digitalRead(buttonPin) == LOW){ //checks if button is pressed --- low meaning pressed

  if(default_alarm != 0){ //so if r1 or r2 is selected
    //call function no.2
    playMusic(melody, durations, sizeof(durations)/sizeof(int));
  }

  else{
    //call other function for barking playback --- function included in the PCM library
    startPlayback(sample, sizeof(sample));
  }

}
```

*Fig 4.30 - Playing Alarm Sound on Button Press

By using a 'for' loop for the button press, we check the value of the variable default_alarm - if it is not 0, meaning that r1 or r2 is selected, Function no.2 (playMusic) executes. If it is 0, meaning that r0 was selected, the startPlayback function runs which is a built-in function that comes with the PCM library we included earlier.

III. Functions Breakdown

Function no.1. - melodyFunction

The purpose of this function is to populate the initially empty melody and duration arrays with the selected audio's melody and duration arrays so that later they can be used for playback in Function no.2.

```
//here goes function no. 1 --- if r1 or r2 was pressed, then the melody[] and durations[] get populated
void melodyFunction (default_alarm, melody_sw, durations_sw, melody_spr, durations_spr, melody, durations, sizeof(durations)/sizeof(int));
```

```
//function no.1: --- for populating arrays

void melodyFunction (int variable, int *source1, int *source2, int *source3, int *source4, int *destination1, int *destination2, int size){


  if(variable == 1){ //if r1 is selected
    for (int i = 0; i < size; i++){
      destination1[i] = source1[i];
      destination2[i] = source2[i];
      }
    }

  if(variable == 2){ //if r2 is selected
    for (int i = 0; i < size; i++){
      destination1[i] = source3[i];
      destination2[i] = source4[i];
      }
    }


}
```

*Fig 4.31 - Populating Melody and Duration Arrays

This function uses the default_alarm variable in main to determine if the chosen system alarm is r1 or r2, and then based on that, picks which arrays should be copied into the melody and duration arrays.

Function no.2. - playMusic

The purpose of this function is to play the melodies of r1 or r2 if one of them is selected as the default alarm sound. This function is called in main when the motion sensor is triggered - in this test version of the code, it is included in a button press loop.

```
if(default_alarm != 0){ //so if r1 or r2 is selected
  //call function no.2
  playMusic(melody, durations, sizeof(durations)/sizeof(int));
}
```

*Fig 4.32 - Playing the Selected Melody

```
//function no.2: --- for playing notes based on arrays

void playMusic(int *melody, int *durations, int size){

  int size = sizeof(durations)/sizeof(int);

  for (int note = 0; note < size; note++){
    //to calculate the note duration, take one second divided by the note type.
    //e.g. quarter note = 1000 / 4, eighth note = 1000/8, etc.
    int duration = 1000 / durations[note];
    tone(BUZZER_PIN, melody[note], duration);

    //to distinguish the notes, set a minimum time between them.
    //the note's duration + 30% seems to work well:
    int pauseBetweenNotes = duration * 1.30;
    delay(pauseBetweenNotes);

    //stop the tone playing:
    noTone(BUZZER_PIN);
  }

}
```

*Fig 4.33 - Melody Playback Using Arrays and tone()

Function no.3. - startPlayback

This is a function included in the PCM library, therefore there is no need to define it separately outside of the main loop. The function is called if the motion sensor is triggered and if it is r0 that was chosen as the default alarm sound. It plays back the encoded audio - in this case, the barking.

```
else{
  //call other function for barking playback --- function included in the PCM library
  startPlayback(sample, sizeof(sample));
}
```

*Fig 4.34 - Playing Encoded Barking Sound with PCM

4.7. LDRs

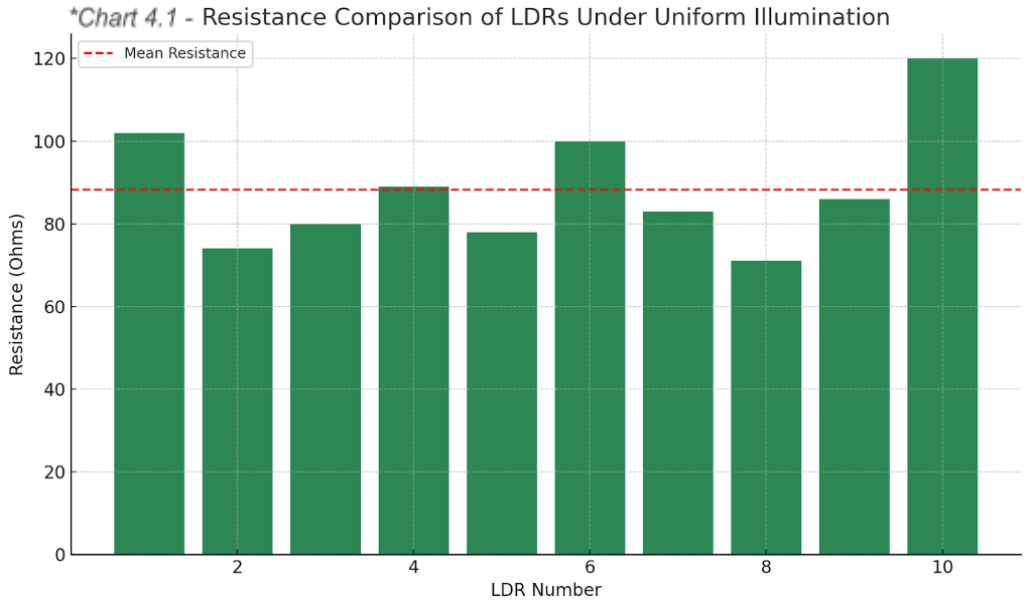
For the light-tracking system, we started with the backbone - the Light Dependent Resistors (LDRs). These sensors are responsible for detecting the direction of incoming sunlight, allowing the solar petals to adjust their orientation for maximum exposure. Since the position with the highest light intensity corresponds to the highest resistance value from the LDRs, precise and consistent readings are critical for reliable tracking. However, because we sourced the LDRs at a relatively low cost, we could not rely solely on the manufacturer's

specifications. This made it essential to carry out our own tests to assess the consistency and performance of each sensor before integrating them into the final system.

To evaluate the reliability and uniformity of our Light Dependent Resistors (LDRs), we conducted a controlled test to measure their resistance under identical lighting conditions. Each of the ten LDRs was exposed to a constant light source through a flashlight positioned almost flush against the sensor surface and held perpendicular to minimize any angular deviation. This ensured that every LDR received the same intensity and direction of light. Each sensor was placed individually onto a breadboard, and we used a multimeter to measure its resistance while carefully controlling for external variables such as ambient light or sensor positioning.

The goal here was not to reach a specific resistance value; instead, we were looking for consistency across sensors. What mattered more for our application was precision rather than absolute accuracy; essentially, how similarly each LDR responded to the same light input, even if that response was slightly offset from a theoretical "true" value. This approach made sense because later in the code the system determines direction based on the relative differences in LDR readings. As long as each sensor consistently responds to light in a similar way, giving similar resistance values, even with a fixed offset, the system can still compare the readings and make correct adjustments.

By identifying and selecting the LDRs with the most closely matching resistance values, we could ensure that any variations in future readings would be due to actual differences in light intensity, not inconsistencies between sensors. This precision is critical for the system's ability to track the sun over time, since what ultimately matters is the difference in readings across the LDRs.



Out of the ten LDRs, four stood out for having very similar resistance readings under identical lighting conditions: LDRs 3, 5, 7, and 9. These were selected for integration into the

tracking circuit. Choosing LDRs that behave similarly under the same light helps minimize errors in directional tracking, making the system more responsive and accurate.

5. Final Code and Assembly

This section outlines the final implementation of both the control code and the mechanical integration that brought our S.O.B. to life. After extensive prototyping and testing, the finalized code was developed to handle tasks like dual-axis motor control, light intensity tracking, motion detection, EEPROM data storage, and system monitoring via a display interface. These functionalities were carefully programmed to run efficiently and autonomously using a loop-based logic structure. Speaker and Nextion display code was also implemented. Simultaneously, attention was given to the physical construction of the product; soldering circuits, replacing fragile wires, organizing components, and resolving structural issues, to ensure the system could operate reliably in real-world conditions.

5.1. Overview of Final Product Function

Our goal was to adjust our solar panels to capture the maximum amount of energy from sunlight. To achieve this, we designed a dual-axis mechanism, using one DC motor for each axis to move the panels in two directions. The code incorporates several fundamental concepts such as PWM, ADC, interrupts, EEPROM usage, the controlling of motors, tracking light, detecting motion, and storing data, making sure the solar panels always face the brightest light source.

5.1.1. Our S.O.B.’s Desired Functionalities

- Moves two DC motors to adjust the solar panel’s position for maximum light.
- Uses light sensors (LDRs) to measure light intensity.
- Tracks motor angles with potentiometers.
- Saves the best light values in the microcontroller’s memory (EEPROM).
- Detects motion with a sensor and logs when motion happens.
- Sends information about the system’s actions to the Nextion Display.

5.1.2. Overall Breakdown of Code

When the system starts:

- It sets up the components.
- It reads the starting angles of the motors from the potentiometers, which tell us the position of each axis (minimum 0° and maximum 300° - angle limits of the 10K potentiometer we are using).
- It checks the microcontroller’s memory (EEPROM) for the best light values from previous runs.
- It prints a countdown from 5 to 1 to show the system is ready to start.

I. Controlling the Motors

The system controls two DC motors, one for each axis of the solar panel:

- Motor A moves one axis and is connected to two light sensors (LDR1 and LDR2) and a potentiometer.
- Motor B moves the other axis and is connected to two other light sensors (LDR3 and LDR4) and another potentiometer.
- Each motor can move forward or backward, and its speed is controlled using PWM.

II. Light Tracking

To find the brightest light, the system moves each motor in a specific pattern:

Motor A

1. Moves 50° forward from its starting position (up to a maximum of 300°).
2. Measures the average light from LDR1 and LDR2.
3. Moves 100° back (down to a minimum of 0°).
4. Measure the light again.
5. The values are stored in the EEPROM and the position with the higher light reading is picked as the target angle.
6. If this light is 5% better than the value stored in memory, it saves the new value in EEPROM.
7. Moves the motor to the position with the best light.

Motor B

Works on the same principle, using LDR3, LDR4, and its own potentiometer.
The potentiometers help the system know the exact angle of each motor, ensuring it stops at the right position.

III. Saving the Best Light Values

The system uses EEPROM to store the highest light intensity values for each motor. After each sweep, it compares the new light reading to the saved one. If the new reading is at least 5% better, it updates the EEPROM with the new value. This ensures the system remembers the best light positions even if it's turned off.

IV. Motion Detection

A motion sensor is connected to the system. When it detects motion, it sends a message like "MOTION: DETECTED". This feature is used to detect if something moves near the SOB and activate our speaker.

V. Serial Monitor Communication

The system sends messages to a computer via serial communication, which would not be relevant while the project works autonomously.

- It reports motor movements (e.g., “Motor A moving forward”).
- It shows the current motor angles (e.g., “Pot A angle: 123°”).
- It displays light levels from the LDRs (e.g., “LDR Avg: 12”).
- It logs motion events with timestamps.

These messages help us monitor what the system is doing while testing and check if it’s working correctly.

VI. Continuous Operation

The system runs in a loop:

- It sweeps Motor A to find the best light position.
- Then, it sweeps Motor B to find the best light position.
- It waits 10 minutes before starting the next cycle. Written as 5 seconds in the logic diagram.

This keeps the solar panel aimed at the brightest light source at all times.

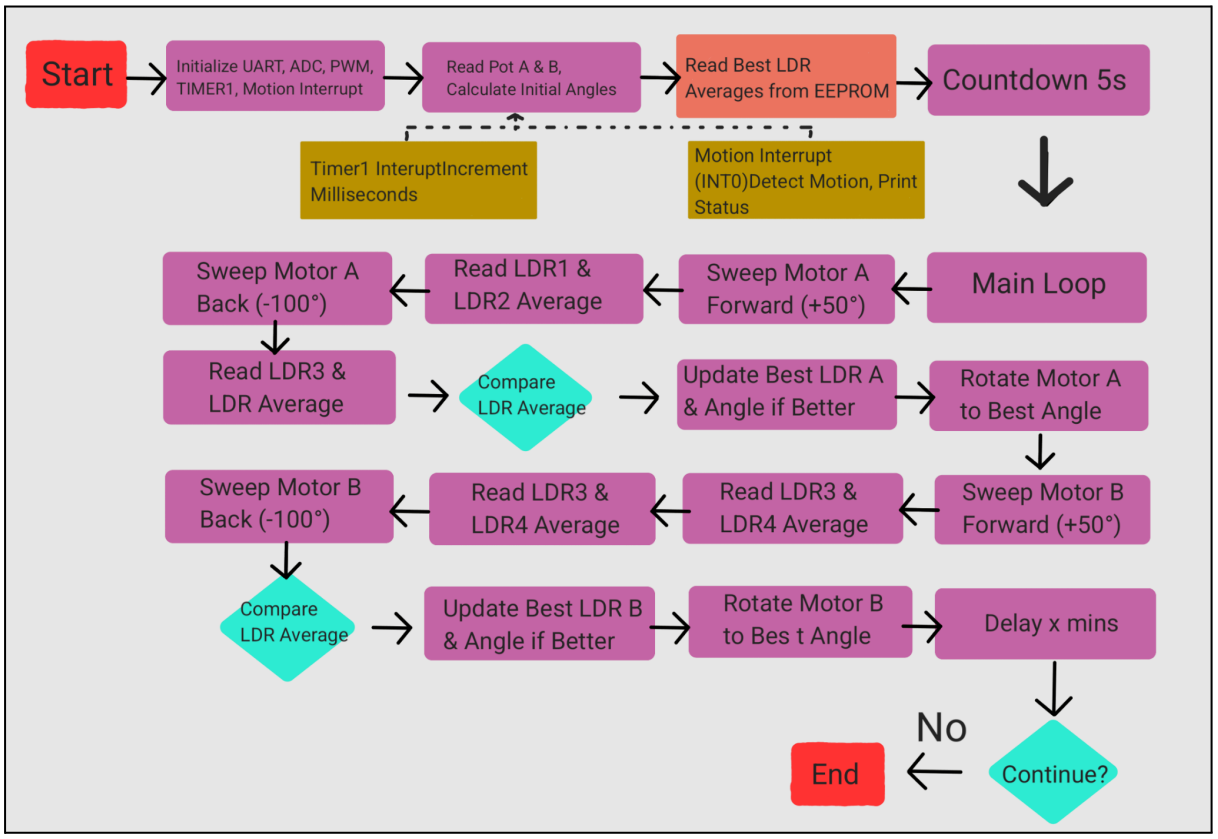


*Fig 5.1 - Flow Chart of Main Code Structure

5.1.3. Detailed Code Breakdown - Setup, Functions, Loops

I. LDRs

Since the readings from the two LDR sensors can produce an average that includes a decimal component (e.g., (375 + 208) / 2 = 291.5), it is necessary to store this result as a float in EEPROM. A float value mostly, including Arduino, occupies 4 bytes of memory. Therefore, the EEPROM addresses for storing the average values are defined 4 bytes apart (e.g., 0x00 for Motor A and 0x04 for Motor B). This separation ensures that the stored data for one set of LDRs does not overlap or corrupt the data from the other set, thereby preventing potential data error issues.

```
#define EEPROM_A_ADDR 0x00 // EEPROM address for Motor A's best LDR average
#define EEPROM_B_ADDR 0x04 // EEPROM address for Motor B's best LDR average
```

**Fig 5.2 - LDR Sensors*

Average light intensity function (LDRs related)

```
// === LDR A/B Averages ===
float ldrA_avg(void) {
    return (analog_read(LDR1) + analog_read(LDR2)) / 2.0f;
}
float ldrB_avg(void) {
    return (analog_read(LDR3) + analog_read(LDR4)) / 2.0f;
}
```

**Fig 5.3 - Light Intensity Function*

These two functions compute the average light intensity sensed by two pairs of Light Dependent Resistors (LDRs) placed onto the ends of the opposite solar panel. The combined LDR values (average) are then labelled A and B for the corresponding motors. This is done to smooth out local lighting variations and produce more reliable light measurements, typically used to make decisions like solar panel alignment, object tracking, etc.

II. Functions

For convenience, all functions were defined prior to the *main()* function. This approach eliminates the need for separate function prototypes, thereby reducing code redundancy. Additionally, defining functions before *main()* improves code readability by allowing the reader to understand the implementation details of each function before encountering their invocations within the main execution flow.

Timer function

```
36  // === millis() timer setup ===
37  volatile uint32_t milliseconds = 0;
38
39  void timer1_init(void) {
40      TCCR1B |= (1 << WGM12);                  // CTC mode
41      TCCR1B |= (1 << CS11) | (1 << CS10);     // Prescaler 64
42      OCR1A = 249;                             // 16MHz / 64 / 1000 - 1 = 249
43      TIMSK1 |= (1 << OCIE1A);                 // Enable Compare A Match interrupt
44      sei();                                   // Enable global interrupts
45  }
46
47  ISR(TIMER1_COMPA_vect) {
48      milliseconds++;
49  }
50
51  uint32_t millis(void) {
52      uint32_t ms;
53      cli();
54      ms = milliseconds;
55      sei();
56      return ms;
57  }
58
```

Fig 5.4 - Timer Function

This code sets up Timer1 to create a millisecond timer. This allows us to track elapsed time using the function *millis()*. We chose to use this function instead of the simple *_delay_ms()* because *_delay_ms()* is a blocked delay function (microcontroller stops for a specified interval of time). Meaning, no sensors or interrupts would work - a challenge we encouraged in the early stages of the coding. On the other hand, *millis()* is a non-blocked timer counter. It just returns the number of milliseconds since the timer started. Therefore, it can be used to check elapsed time without stopping the microcontroller, allowing the program to continue running other tasks simultaneously.

Line 40: Sets bit WGM12 to 1, hence, puts Timer1 in CTC mode.

Table 17-5. Waveform Generation Mode Bit Description(1)

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x	TOV1 Flag Set on
4	0	1	0	0	CTC	OCR1A	Immediate	MAX

Table 5.1 - Waveform Generation Mode Bit Description [3]

Line 41: The data used to set the prescaler to the 64 - data from the ATmega32A Datasheet.

17.10.2 TCCR1B – Timer/Counter1 Control Register B

Table 17-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$clk_{I/O}$/1 (No prescaling)
0	1	0	$clk_{I/O}$/8 (From prescaler)
0	1	1	$clk_{I/O}$/64 (From prescaler)
1	0	0	$clk_{I/O}$/256 (From prescaler)
1	0	1	$clk_{I/O}$/1024 (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Table 5.2 - TCCR1B - Timer/Counter1 Control Register B [3]

Line 42: In CTC mode, Timer1 counts up from 0 to the value in OCR1A (249), then resets to 0 and triggers an interrupt. The clock frequency is 16MHz, Timer1 prescaler set to 64, 1ms is 1/1000 of a second.

$$Timer\ frequency = \frac{CPU\ Frequency}{Prescaler} = \frac{16000000}{64} = 250000\ ticks\ per\ second$$

$$Ticks\ per\ 1\ ms = \frac{Timer\ frequency}{1000} = \frac{250000}{1000} = 250\ ticks$$

$$Since\ the\ timer\ counts\ from\ 0\ upto\ the\ OCR1A\ value,$$

$$OCR1A + 1 = 250$$
$$OCR1A = 250 - 1 = 249$$

**Fig 5.5 - OCR1A Value*

Line 43: For Timer1, since we are using OCR1A in Line 42, we need to find its corresponding interrupt enable - which is OCIE1A. The TIMSK1 is the Timer/Counter Interrupt Mask Register. The OCIE1A interrupt enabling is done by setting this bit to 1.

Line 47: Interrupt Service Routine (ISR) is a special function that is automatically executed by the microcontroller when a specific interrupt occurs (in this case, triggered by Timer1). This function is called every 1 millisecond due to Timer1 running in CTC mode.

TIMER1_COMPA_vect is the predefined name used to create an ISR for the Timer1 Compare Match A interrupt. So when the interrupt is triggered, the milliseconds variable will increase by 1 millisecond every 1ms.

Interrupts briefly pause the main program to run a small, fast piece of ISR code - in this case, incrementing a millisecond counter every 1 ms. This pause only lasts a few microseconds, allowing precise timing without blocking or slowing down the main code. Since the ISR is short, in theory, the program keeps running smoothly almost all the time.

Because we would like the code to run in an interval we set (so that we are not constantly changing the solar panel position - only when significant changes to light intensity is expected), we need to implement this timer code.

Line 51 - 56: These lines together return the current value of milliseconds, which is a global variable incremented every 1 ms by the Timer1 ISR. *cli()* and *sei()* are simply predefined functions in C that allow to enable and disable interrupts.

ADC reading and configuration function

This function configures the ADC input channel and reference voltage, starts an ADC conversion, waits for it to finish, and returns the digital value representing the analog voltage

on the chosen channel. The function is important for our application as we would like to convert the analog signals - voltage levels coming from our LDRs and potentiometer - to digital data the program can use. The program uses the digital data to interpret the environment and make decisions accordingly. For example, it reads values from an LDR (analog converted to digital data) to determine how bright or dark the surroundings are. Based on this information, the system can respond appropriately - in our case, by sending signals to the motors to adjust the position of the solar panels for best energy generation.

```
76  // === ADC Read ===
77  uint16_t analog_read(uint8_t channel) {
78      ADMUX = (1 << REFS0) | (channel & 0x07);
79      ADCSRA |= (1 << ADSC);
80      while (ADCSRA & (1 << ADSC));
81      return ADC;
82  }
```

Fig 5.6 - ADC Reading and Configuration Function

Line 77: Defines a function that takes an 8-bit unsigned integer variable called channel (which ADC input to read - defined later).

Line 78:
Sets the reference voltage for ADC to AVcc. REFS0 bit = 1 means AVcc is used as the ADC voltage reference.

23.9.1 ADMUX – ADC Multiplexer Selection Register

Bit	7	6	5	4	3	2	1	0	
	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 5.3 - ADCMultiplexer Selection Register [3]

Line 79: ADCSRA is the ADC Control and Status Register A. ADSC (ADC Start Conversion) bit is set to 1 here. This is what initiates the ADC conversion process. The OR bitwise operator is used so that we only set the ADSC bit, leaving other bits unchanged.

Line 80 - 81: The ADSC bit remains set to 1 while conversion is in progress and resets to 0 when the conversion is complete. The loop exits once ADSC becomes 0, indicating the ADC result is ready.

III. Motor Control Breakdown

Output pins configuration for motor control

The DDRD register is configured to set specific pins as outputs, which control the direction pins for two motors. ADC functionality is initialized with a prescaler of 64 to balance speed and resolution, enabling accurate analog readings from potentiometers and LDR sensors.

The *pwm_init()* function sets up Timer1 for fast PWM operation to control motor speed, while *timer1_init()* configures the same timer for CTC mode to support the *millis()* timing function - although using Timer1 for both purposes introduces a known conflict that has been discussed previously in point number 8.

```
151     DDRD |= (1 << IN3) | (1 << IN4) | (1 << IN1) | (1 << IN2);
152     ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1);
153     pwm_init();
154     timer1_init();
155     motion_interrupt_init(); // Initialize motion sensor interrupt
156
157     angleA = (analog_read(potA) * 300.0f) / 1023.0f;
158     angleB = (analog_read(potB) * 300.0f) / 1023.0f;
159
160     float bestA = eeprom_read_float((float*)EEPROM_A_ADDR);
161     float bestB = eeprom_read_float((float*)EEPROM_B_ADDR);
162
163     printf("Starting system...\r\n");
164     for (int i = 5; i > 0; i--) {
165         printf("%d...\r\n", i);
166         _delay_ms(1000);
167     }
```

**Fig 5.7 - Output Pins Configuration for Motor Control*

The current angles of both motors are calculated from analog potentiometer readings and converted from ADC values to degrees (0-300°), reflecting the limits of the 10k potentiometers used. Calibration values or previously recorded optimal LDR averages are retrieved from EEPROM to maintain system performance across reboots. Finally, a startup countdown provides a short delay for system stabilization and user feedback, ensuring reliable initial operation. This initialization process ensures that all hardware components are properly configured before entering the main while loop.

Motor A and B sweeps - for continuous solar tracking

The primary purpose of this continuous loop (while(1)) is to implement an autonomous, closed-loop solar tracking system using two independent motors - Motor A and Motor B. Almost the same sweep function is used for both Motor A and B - however, they differ by the use of different variables and registers used. These motors adjust the orientation of solar panels based on light intensity feedback from the LDRs. By regularly sampling light levels and adjusting motor angles, the system ensures that the panels are consistently aligned with the direction of maximum light, thereby optimizing energy collection.

```
169     while (1) {
170         // === Sweep Motor A ===
171         float a_forward = angleA + 50.0f; if (a_forward > 300.0f) a_forward = 300.0f;
172         rotate_motor("A", "A", a_forward, &angleA, potA, motorA_set_direction, motorA_set_speed, motorA_stop, ldrA_avg);
173         float avgA_f = ldrA_avg();
174 
175         float a_back = angleA - 100.0f; if (a_back < 0.0f) a_back = 0.0f;
176         rotate_motor("A", "A", a_back, &angleA, potA, motorA_set_direction, motorA_set_speed, motorA_stop, ldrA_avg);
177         float avgA_b = ldrA_avg();
178 
179         float best_avgA = (avgA_f > avgA_b) ? avgA_f : avgA_b;
180         float best_angleA = (avgA_f > avgA_b) ? a_forward : a_back;
181         if (best_avgA > bestA * 1.05f) {
182             bestA = best_avgA;
183             eeprom_write_float((float*)EEPROM_A_ADDR, bestA);
184         }
185         rotate_motor("A", "A", best_angleA, &angleA, potA, motorA_set_direction, motorA_set_speed, motorA_stop, ldrA_avg);
186 
187         // === Sweep Motor B ===
188         float b_forward = angleB + 50.0f; if (b_forward > 300.0f) b_forward = 300.0f;
189         rotate_motor("B", "B", b_forward, &angleB, potB, motorB_set_direction, motorB_set_speed, motorB_stop, ldrB_avg);
190         float avgB_f = ldrB_avg();
191 
192         float b_back = angleB - 100.0f; if (b_back < 0.0f) b_back = 0.0f;
193         rotate_motor("B", "B", b_back, &angleB, potB, motorB_set_direction, motorB_set_speed, motorB_stop, ldrB_avg);
194         float avgB_b = ldrB_avg();
195 
196         float best_avgB = (avgB_f > avgB_b) ? avgB_f : avgB_b;
197         float best_angleB = (avgB_f > avgB_b) ? b_forward : b_back;
198         if (best_avgB > bestB * 1.05f) {
199             bestB = best_avgB;
200             eeprom_write_float((float*)EEPROM_B_ADDR, bestB);
201         }
202         rotate_motor("B", "B", best_angleB, &angleB, potB, motorB_set_direction, motorB_set_speed, motorB_stop, ldrB_avg);
203 
204         _delay_ms(5000); // wait before next cycle
205     }
```

Fig 5.8 - Motor A and B Sweeps

For each motor, the loop executes a sweep routine. It first calculates a forward angle (+50° from the current angle, clamped to a maximum of 300°) and rotates the motor to this position using the generalized *rotate_motor()* function. This function takes in various control and sensor function pointers, allowing reuse for both motors. Once in position, the system records the light intensity from the associated LDRs. Then, a backward sweep (-100°, clamped at 0°) is executed in the same manner, and the light intensity at that position is also recorded. By comparing the forward and backward LDR readings, the system determines the angle that yielded better exposure measured during the sweep. This is first executed for Motor A. Once the best angle of most light intensity is found and adjusted to it, the same pattern is repeated for Motor B. The motors control movement in different axes in the 3D plane.

If the better reading exceeds the previously stored best value in EEPROM by more than 5% - a condition we implemented to prevent movements for very little changes to light intensity so that the system is not constantly moving - the system updates this value in non-volatile memory and moves the motor to the corresponding optimal angle. The pointer &angleA or &angleB allows the *rotate_motor()* function to update the current motor angle in-place after reading the potentiometer feedback (converted from a 10-bit ADC value to a 0-300° angle). This real-time update is essential for reevaluating the loop condition and ensuring accurate motion control.

A delay of 5 seconds at the end of each loop start allows for system stabilization and prevents excessive motor activity, which can lead to mechanical degradation or power waste.

5.2. Construction and Circuit Assembly

Once testing and validation were complete, we soldered all electrical connections onto prototyping boards. Each LDR was wired into a voltage divider circuit with a 100-ohm resistor going to ground. The analog output from each LDR was connected to an analog input pin on the Arduino Nano, allowing us to continuously monitor light levels.

During soldering, we were careful to avoid cold joints and ensured all connections were mechanically and electrically sound. However, the use of extremely thin copper wires for much of our internal wiring posed a serious issue. As on the day we needed to complete our connections, the electronics lab was closed, we were left with limited resources. Although these wires allowed us to proceed, they were not ideal because they frequently broke and created unreliable connection points due to their fragility. Eventually, we had to replace many of them with sturdier wires and re-solder key joints to ensure stable electrical connections.

We ended up soldering the battery circuit twice because the first time one of the wires burned from the inside. Since it was the positive wire coming directly from the battery, it is possible the problem was caused partially by a bad solder joint and partially by the weakness of the material, which caused the wire to overheat. Luckily, we were there when it happened and cut the wire so the others were not damaged. We replaced the faulty wire with a better-quality one and re-soldered everything properly to make sure it was safe and working as it should.

During the re-soldering process, we routed the wires through the stem of the structure. Although mechanically straightforward, the task was complicated by the presence of 20 wires of the same color making it challenging to distinguish their individual functions. To resolve this, each wire was labeled using adhesive tape with identifiers, allowing for precise mapping of connections throughout the circuit. This labeling method significantly improved clarity and reduced the likelihood of miswiring during final assembly.

Once all solder joints were completed and connections secured, we threaded the fishing line through the stem to mechanically link the system. Each line was attached to a solar panel arm at the top and tensioned at the base, where it was securely tied to the corresponding potentiometer gear. Each pair of opposing solar panel arms was connected to a single gear driven by a dedicated servo motor, ensuring synchronized motion and structural balance.

In addition, another challenge emerged during the final assembly stage. The ball that supported the ball socket for the solar panel and LDR assembly proved to be a structural weak point and, unfortunately, broke. Instead of reprinting the part, which would have taken several hours, we used a soldering rod to carefully melt and fuse the surrounding PLA. This improvised solution successfully reinforced the joint, allowing us to move forward without significant delays while still maintaining the integrity of the design.

6. Project Evaluation and Conclusion

This final chapter contains our reflections on the overall success of the project based on our predefined requirements and objectives. We also briefly discuss the somewhat more subjective aspects of the project: teamwork, collaboration and our group dynamic in general.

6.1. Other Challenges and Learning Outcomes

6.1.1. Challenges with the Speaker and the Alarm

The first challenge that we faced relating to the speakers was that our first test circuit was incorrect. This set the speaker coding back by about a day, as we had to go over it again and figure out what caused the issue. Eventually, it turned out that the one video we chose from about the 5-10 we had watched, was the one with the circuit set up the wrong way. Once we switched the wires, it was alright though, and we could move on to the Nextion-Arduino-speaker code. The main takeaway from this would probably be to not only study multiple resources, but actually compare them to see if there are any major differences that could account for the encountered problems.

The next difficulty was with setting up the Nextion to Arduino communication. This took quite a while as most resources focused on setting up the Nextion so that it could read commands from the Arduino, not the opposite way around. The final code for this was assembled from one tutorial[11] in particular.

Not only was it challenging to understand this setup, but the code itself proved to be really complicated to write. The final result seems quite clear and easily understandable to the reader, but figuring out its structure took a lot of time. This was due to multiple factors: the two different types of playback being used, meaning that we needed different functions for them; getting any value from the checkboxes that could be read in the main loop; deciding where and how to use the functions - what we should include in them, what could only be outside of them, etc. as well as how many functions to use.

A minor issue also occurred during the last week of the project period: namely, due to a short circuit, the Nextion display we were using got fried. Supposedly, we were using it in the same circuit setup as we previously built - one that was fully functional and had no such problems previously. Luckily, our professors could provide us with a new one on a pretty short notice, so it was only for a weekend that we could not run anything on it.

The last problem we encountered was that during the many testings and circuit assemblies both of our speakers broke, pending the testing of the Nextion-Arduino-speaker code. This was more of an annoyance than a critical issue, as even though the code was operational in theory, we had no way of knowing if it would actually execute the way we wanted it to. We did not spend our time sitting around, just waiting for the new speakers, of course, as there were plenty of other tasks to attend to - soldering the final circuit, writing the report, finishing up the rest of the code, doing the designs - but it was nonetheless very stressful only a few days before the deadline.

6.1.2. Challenges with Designing and Printing

One of the biggest challenges we faced during the design and 3D printing process was making sure all the parts were the right size and would fit together properly. We had to take a lot of careful measurements, especially since different components - like the stem, pot, and solar panel arms - had to connect and align with each other. If even one part was slightly off in scale, it could throw off the whole assembly. We also had to pay close attention to proportions to make sure the final product looked balanced and nothing appeared too big or too small.

Printing the parts came with its own set of issues and was by far the most time-consuming part of the project. On average, each print took about an hour, and we often had to just hope that the printed pieces would match up correctly in real life. It was not always predictable - sometimes things would look perfect in the design software but would not fit or align properly once printed. At a certain stage, we began printing only the fitting components instead of the entire design in order to save time and improve productivity. By utilizing the section cut feature in Bambu Studio, we were able to isolate and print only the relevant fittings.

Technical problems with the 3D printers also slowed us down. There were times when the printers broke mid-print, the filament ran out without warning, or prints failed completely due to errors we could not always control. Each of these problems meant starting over, which used up more PLA and added more hours to our timeline.

These issues were especially frustrating toward the end of the project when we were short on time. To avoid having to reprint every single broken piece, we sometimes used a soldering iron to carefully heat and fuse the PLA parts together. This quick fix worked surprisingly well in most cases and helped us save both time and material without seriously affecting how the parts held up. It turned out to be a useful workaround that kept us on track when reprinting was not a practical option.

6.1.3.Challenges with the Batteries

One of the challenges we faced during the project was building a battery circuit to power all our components. Initially, we soldered everything together (batteries and resistors in circuit and connected to the Arduino Pins and the GND), but just as we were finishing up, one of the wires overheated. It turned out that the wire (connected to the positive terminal of one of the batteries) was made of cheap copper and could not handle the current going through it. Combined with a potentially poor solder joint, the wire began to overheat and burn from the inside. Fortunately, we were present when it happened and quickly cut the wire before the issue could spread or damage other parts of the circuit. After inspecting everything, we replaced the faulty wire with a better-quality one and re-soldered the connections properly, making sure it was safe and secure.

After that, we ran into another issue: even with two batteries, the total voltage was not enough to reliably power all of our components. We did not have any extra input pins available for a third battery, so we had to find a smarter solution using the existing setup. Ultimately, we decided to connect two of the batteries in series. This allowed us to increase the voltage without needing additional pins. However, since the Arduino can only safely read voltages between 0V and 5V, we had to be careful. To protect the microcontroller, we

redesigned the voltage dividers by using new resistor values that would bring the higher voltage safely down to a readable range.

6.2. Evaluation of the Fulfillment of Requirements

Looking back at the project, we are satisfied with how it turned out. Our final product works as expected and meets the goals we set for ourselves. We managed to design a solar-powered, flower-shaped security system that tracks the sun and responds to motion with a sound deterrent - just like we planned in the initial stages of development.

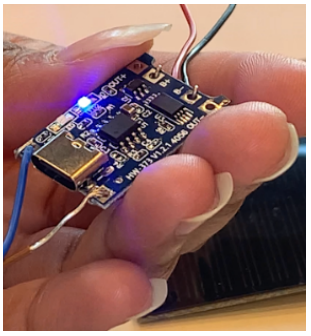
Even though we faced multiple challenges along the way and made some mistakes, we were able to spot and fix them based on priority levels - some were left out, such as Timer1 issues, for future improvements due to time constraints. Each problem became a learning opportunity, and we used those experiences to improve both the product and our understanding.

Overall, we believe we successfully fulfilled our own basic requirements for functionality, creativity, and teamwork. The result is a working and unique prototype that we are proud of.

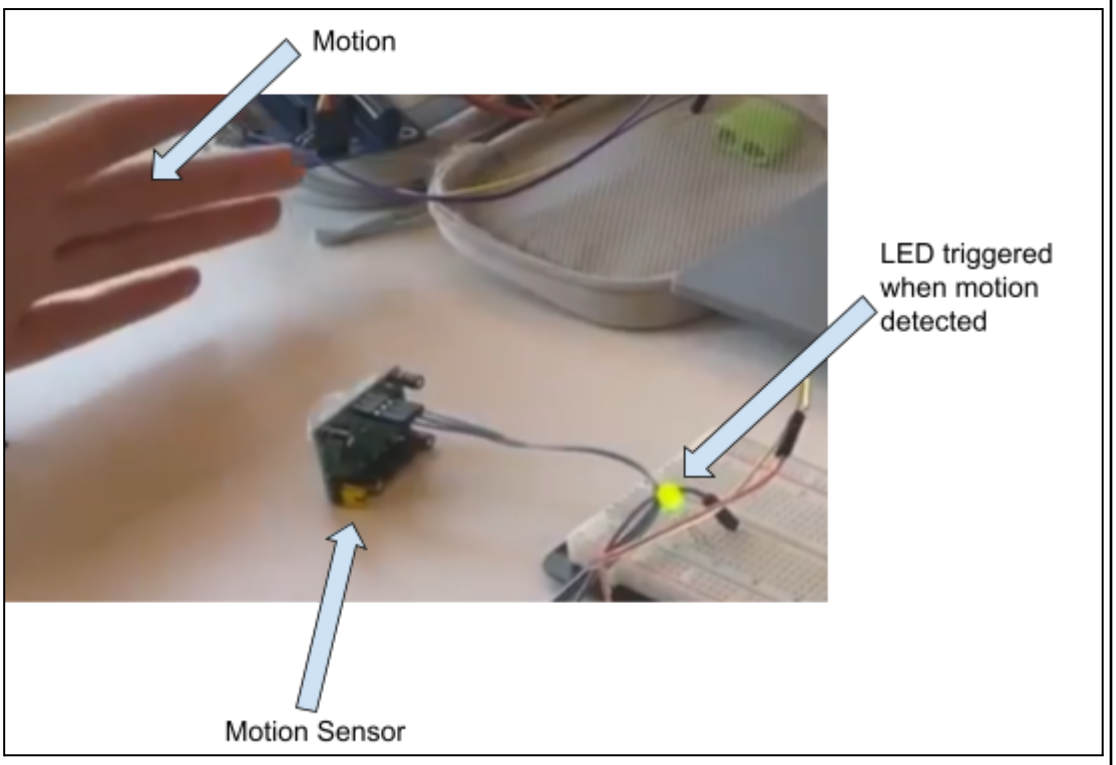
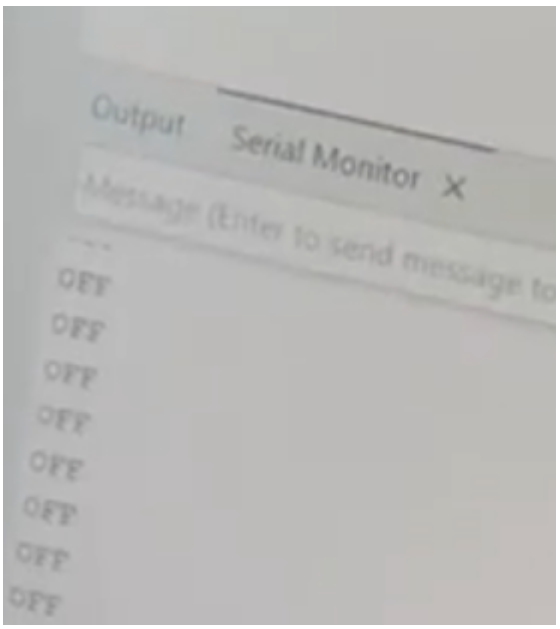
Planned Requirement	**Outcome**

Use servo motors to move solar panels and set solar panel angle for (near) maximum light intensity
 	
Servo movement angle (accurate rotation using potentiometers)
	Achieved. During the testing, we received a target Pot Angle (potentiometer angle) based on the LDR digital signals for the highest light intensity which was fed into the separate motors for angle adjustment. Essentially, two-axis solar tracking was implemented with DC motors and potentiometers for feedback. Functions discussed in *Chapter 4.2.3.* and *5.1.3.III.* helped to implement the potentiometer angles mapped from 0 - 300° (movement accurate within ~±1°).
	
	Snippet of Serial Monitor:
	`[INFO] Rotating Motor A to 239.74° using Pot A`
	`→ Motor A: FORWARD`
	`↳ Pot A angle: 178.30°, LDR Avg: 622.00`

Integrate solar panels for sustainable energy	Successfully added. Panels power the system during the day; batteries store energy for later use. The success of this solar panel + battery circuit working was evaluated using the LED indicators on the Battery Charging Modules; where red LED on meant charging and blue LED meant fully charged battery. The lack of any LED on signified faults in our circuit.

	
Include a barking sound as a security alarm when motion is detected. Alarm volume around 70-80 dB	Implemented using PCM playback of a dog bark file and a working speaker circuit. In fact, we also added other audio files for improved functionality and better user-experience (as they can choose the audio in the library we provide). This was achieved using Nextion buttons and separate playback code for multiple alarm sounds. This has been discussed earlier in *Chapter 4.6.* Barking sound output via speaker + transistor circuit reached suitable loudness. Roughly estimated around 75 dB. Hence, a success.
Create a decorative, flower-inspired design	We sought feedback from others by asking what the overall shape reminded them of, and many correctly identified it as a flower. Based on this response, we consider the design a success in effectively conveying our intended concept.
Add a Nextion display to show system info (specifically battery levels, clock, motion detection time, all through an interactive UI)	Display works and shows battery % and logs motion events with a custom visual interface. This was implemented using voltage dividers; values were read on A6 and A7 analog pins and displayed on the screen. In addition, a software-based clock was used for timing purposes. While a real-time clock (RTC) module would have provided greater accuracy, it was not implemented due to time constraints. However, incorporating an RTC remains an interesting area for future improvement. RTC will allow us to keep the time even when the main microcontroller is powered off - improved accuracy of clock and uses very little power. Furthermore, for our UI, we have 3 buttons on-screen with a nice-looking background to let the user pick between 3 alarm sounds. This has been discussed in *Chapter 4.5.* Due to the working Nextion display with a working feedback loop, we consider this as a success.

Motion detection within a 3–5 m range and activation of an alarm upon motion detection	HC-SR501 sensor worked reliably after testing; basic filtering for false alarms was considered. Upon motion detection during testing, we implemented a feedback look that would output to the serial monitor upon motion detection. This was successful as discussed in *Chapter 4.4.2* and *5.1.2.IV.*
	`if (PIND & (1 << MOTION_PIN)) {`
	`  printf("MOTION: DETECTED at %lu ms\r\n", millis());`
	`} else {`
	`  printf("MOTION: NONE at %lu ms\r\n", millis());`
	`}`

	Additionally, during testing using HC-SR501 PIR sensor, we were reliably able to detect motion up to ~5 meters with ~120° angle. Output on the serial monitor showed accurate results. For testing, we introduced an LED that would turn on when motion had been detected by the sensor. This OFF/ON output was also displayed on the serial monitor.   *Serial Monitor Output when no motion was detected
Keep final assembly under 2 kg and within a 2000 DKK budget	We created an Excel Sheet for the Components and Financing where to take account of all components we purchased. The following is a snippet:

<table>
<tr><td></td><td>
<table>
<tr><th>Initial Budget/
DKK</th><th>Leftover</th></tr>
<tr><td>2000</td><td>730</td></tr>
</table>

Additionally, the product was intentionally designed to be lightweight, allowing customers to easily move and place it wherever needed for security purposes, whilst still being heavy enough to support itself securely. Although we did not have access to a scale to measure the final weight of the assembly, we estimate it to be approximately 1.5 to 1.7 kg. Therefore, the design successfully meets both criteria.</td></tr>
<tr><td>Use low-power mode for energy efficiency. System should work for at least 9 consecutive hours after last charge</td><td>Depends on usage, but worked throughout full testing days; power optimization could extend it further.

Hence, due to the lack of proper testing, we will label this as a partial success. Some components go into low-power mode, but power optimization could still be improved further.</td></tr>
<tr><td>Avoid false triggers from small animals or wind</td><td>This feature was not fully implemented - while the idea was considered, it was not tested due to time constraints. However, it presents an interesting opportunity for future development, as it could significantly enhance the functionality and reliability of our product. Several approaches could be explored to achieve this. One option is to use sensors with reduced sensitivity. Another is to implement software that only triggers the audio output when sustained motion is detected for a specific duration (e.g., 3 seconds). A simple function like the following could be used:

```
#define MOTION_THRESHOLD_DURATION 3000  // 3 seconds for now but can be changed!!

ISR(INT0_vect) {
    // Check if motion signal is high (motion detected)
    if (PIND & (1 << MOTION_PIN)) {
        uint32_t start_time = millis();

        // Wait while motion signal remains high
        while ((PIND & (1 << MOTION_PIN)) && (millis() - start_time < MOTION_THRESHOLD_DURATION)) {
            // Do nothing – just wait for motion to persist
        }

        // Confirm sustained motion
        if ((PIND & (1 << MOTION_PIN)) && (millis() - start_time >= MOTION_THRESHOLD_DURATION)) {
            printf("MOTION: SUSTAINED DETECTION (≥ 3s) at %lu ms\r\n", millis());
            //ADD AUDIO OUTPUT CODE HERE

        } else {
            printf("MOTION: BRIEF / FALSE TRIGGER ignored\r\n");
        }
    }
}
```
</td></tr>
<tr><td>Use Timer1 (16bit timer) for PWM and millis() simultaneously</td><td>Unfortunately, this did not work as intended. We learned that these functions conflict; a workaround was applied, but is not ideal for future application. This was discussed in detail in Chapter 4.2.2. Hence, although the requirement was achieved, it is not a suitable requirement.</td></tr>
</table>

**Table 6.1 - Requirements set by us and their outcome*

6.3. Reflection on Project Group Dynamic and Collaboration

Our project group consisted of seven individuals, most of us already knowing each other at the beginning of the second semester. The dynamic within the group quickly developed to be of an amicable sort as all of the members were enthusiastic about the challenges of the second semester project.

As the initial problem formulation was due shortly after the beginning of the semester, we had to start working together as a team quite early on. Even during the initial conceptualization of our project, it was clear that not only were all of us alike in our mindset and ideation, but we were also highly compatible personality-wise as a group. There were hardly any uncomfortable silences and we soon discovered that we shared similar interests outside of the field of engineering too.

Group hangouts became regular during the months between the submission of the problem formulation and the start of the project period. Of course, we worked on the development of our project too, but most of the time we hung out rather as friends than colleagues. In addition, we took part in the SDU Case Competition this spring both as a great practical learning opportunity and as a team bonding exercise. It was an excellent chance to enhance both our hard and soft skills as a team as well as individuals. We gained a lot of experience in project development and also learned what strengths each of us can bring to the table.

Throughout the project period, our primary objective was to both develop new skills and strengthen our existing ones. To achieve this, we assigned tasks based on individual strengths and interests. Each task was handled by a smaller team of 2–3 members, allowing us to combine our diverse skill sets and provide mutual support. This collaborative structure ensured that no one faced challenges alone and that every aspect of the project benefited from shared expertise.

Overall, we believe that we achieved an efficient and ultimately successful, collaborative team environment where not only could we build our project but lasting friendships too.



Fig.6.2 - Our Team

7. References

Battery University. “BU-302: Series and Parallel Battery Configurations.” Battery University, 3 Mar. 2022, https://batteryuniversity.com/article/bu-302-series-and-parallel-battery-configurations.

Developing in C for the ATmega328P: Function - analogRead() | Wellys Dev. 12 Sept. 2022, https://wellys.com/posts/avr_c_function_analogread.

electronic GURU. “Talking Arduino | Playing MP3 Audio With Arduino | Arduino PCM Audio Without Audio or Mp3 Module.” YouTube, 3 May 2018, https://www.youtube.com/watch?v=F28Znry0qcw.

Electronics BD. “How to Play Audio With Arduino.” YouTube, 4 Oct. 2019, https://www.youtube.com/watch?v=aaqaAXlZbuc.

Hibit-Dev. “Buzzer/Src at Master · Hibit-dev/Buzzer.” GitHub, https://github.com/hibit-dev/buzzer/tree/master/src.

InterlinkKnight. “Nextion+Arduino Tutorial #2 Sending Data to Arduino.” YouTube, 20 Sept. 2017, https://www.youtube.com/watch?v=mdkUBB60HoI.

“iPhone Wallpaper - Sunflowers Illustration.” Pinterest, 25 May 2025, hu.pinterest.com/pin/840976930457870043.

Just Baselmans. “Make Arduino Play ANY Audio File (Without SD Card) QUICK TUTORIAL.” YouTube, 30 Apr. 2021, https://www.youtube.com/watch?v=m1HEwgHSBrs.

Microchip Technology Inc. megaAVR® Data Sheet. DS40002072A, 2018, https://www.microchip.com. Accessed 1 Apr. 2025.

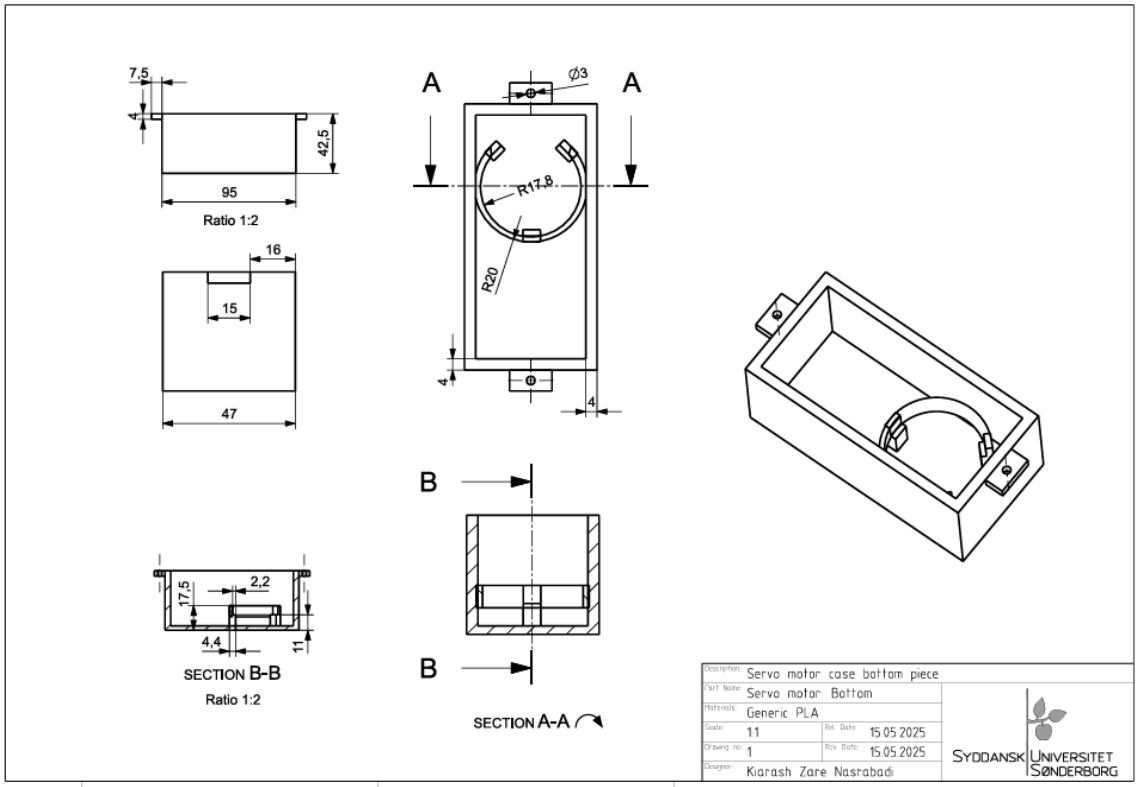
“Playing Popular Songs With Arduino and a Buzzer.” Arduino Project Hub, https://projecthub.arduino.cc/tmekinyan/playing-popular-songs-with-arduino-and-a-buzzer-546f4a.

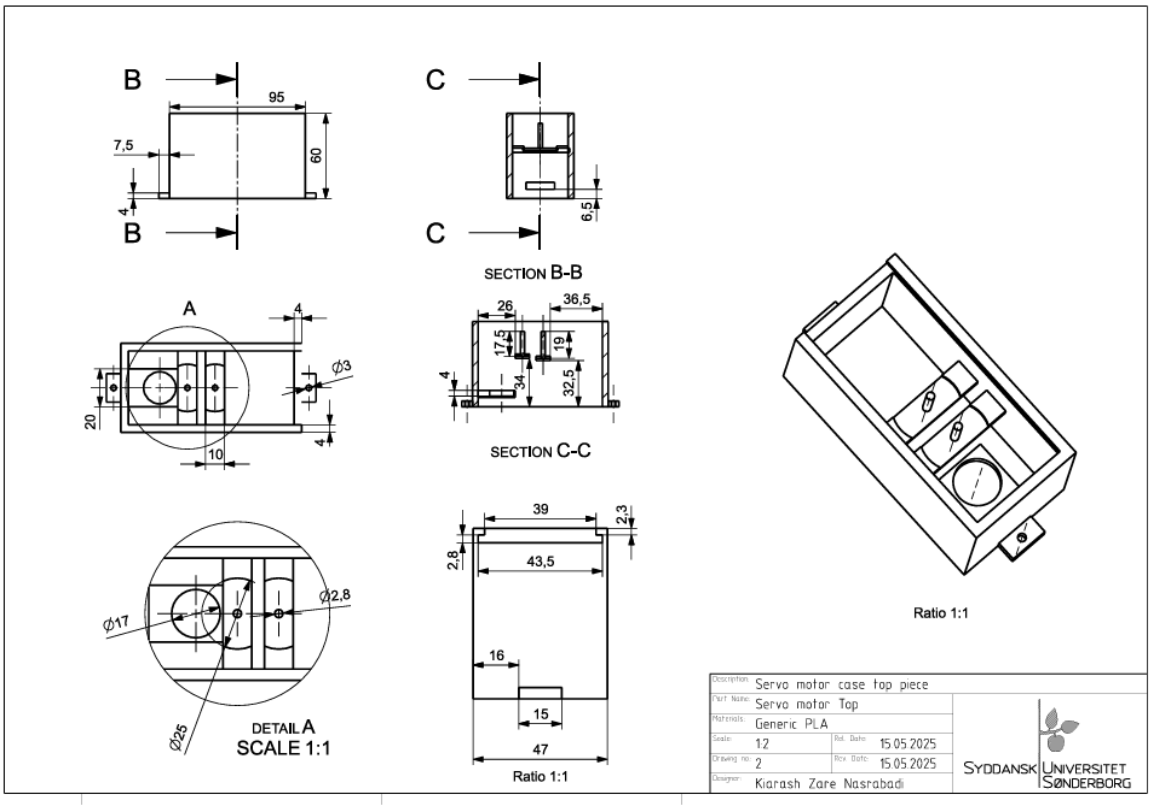
“Rawpixel.” rawpixel.com, www.rawpixel.com/image/558733/sunflower-pattern?referral=103&source=pinterest.

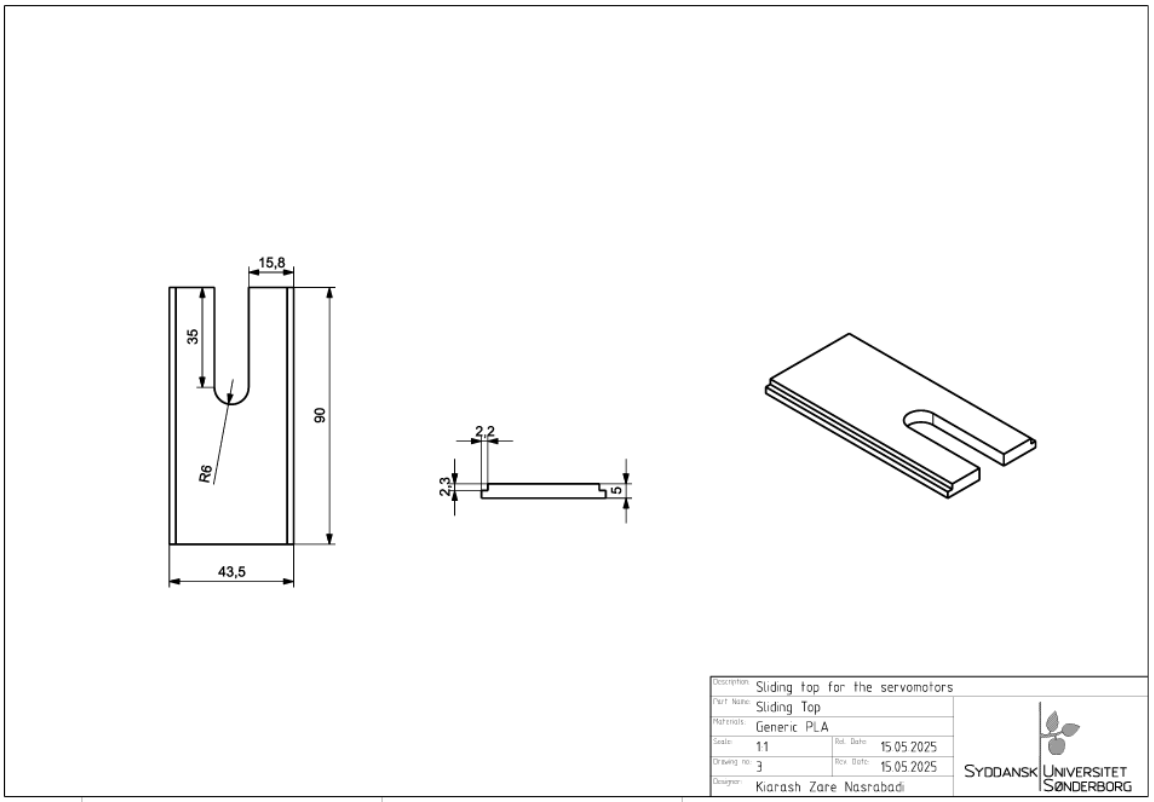
Shveytank. “GitHub - Shveytank/Arduino-MP3.” GitHub, https://github.com/shveytank/Arduino-MP3/tree/master.

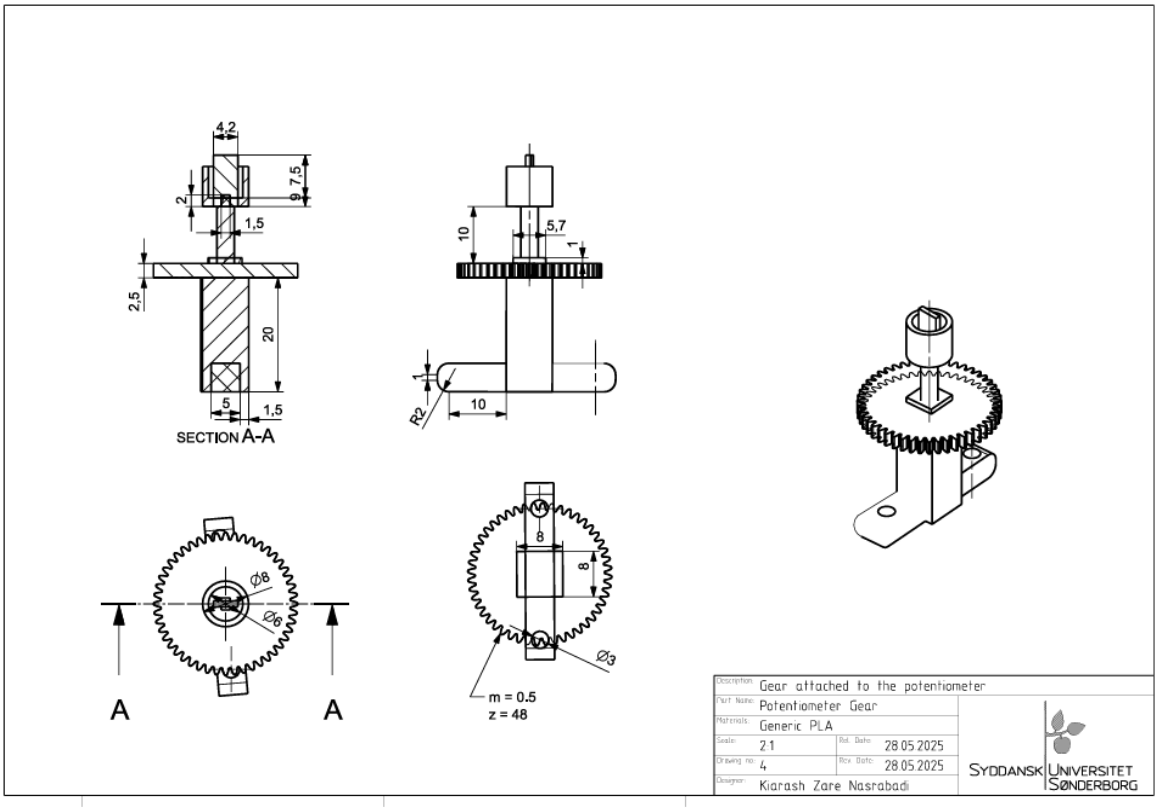
8. Appendix

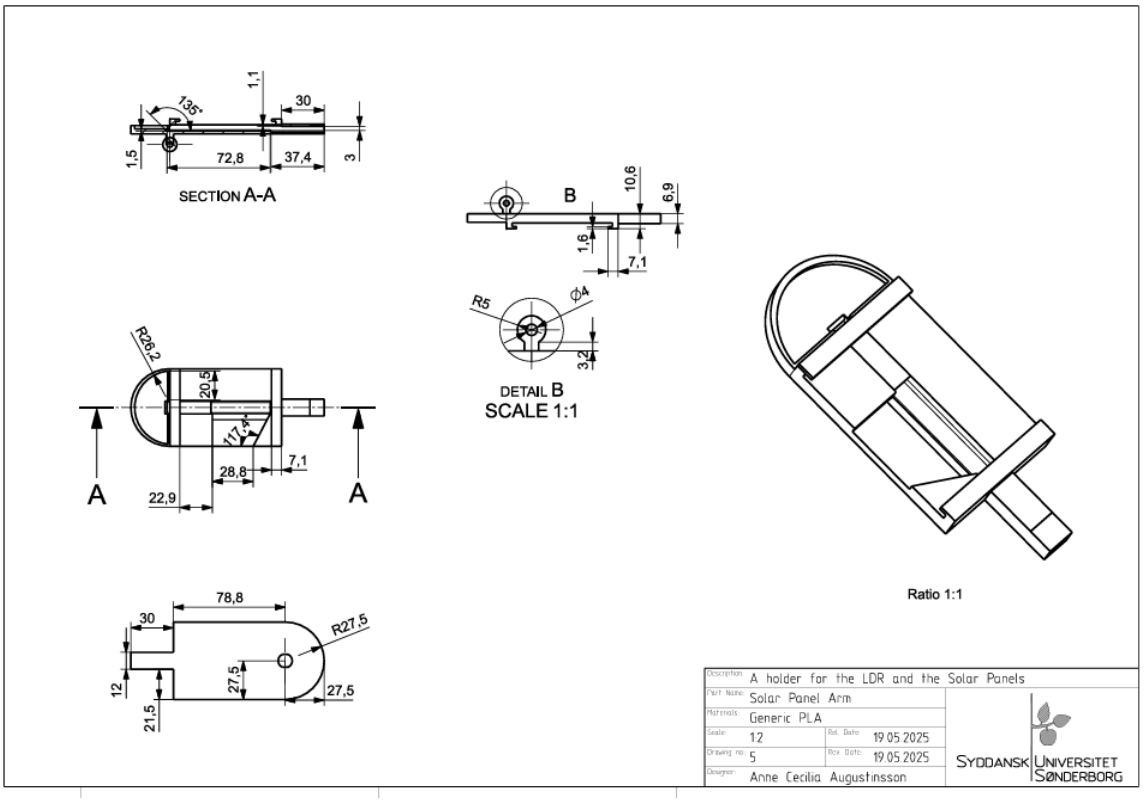
8.1. Machine and Technical Drawings

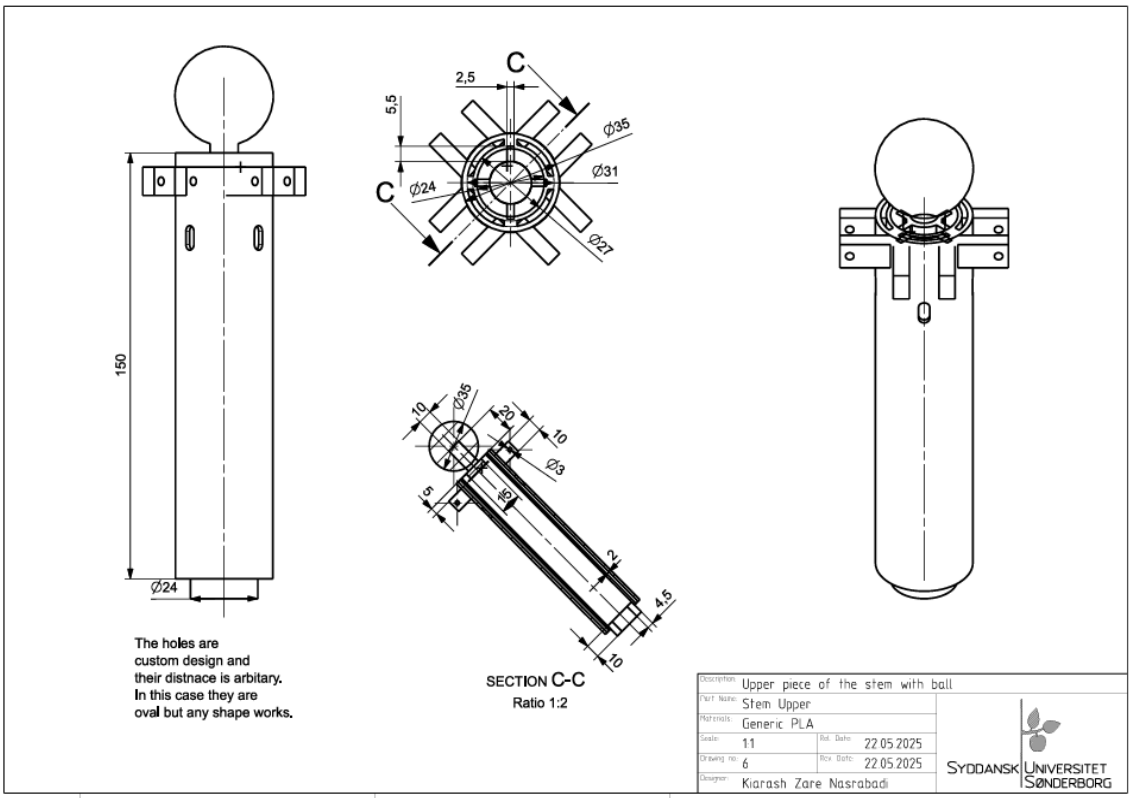


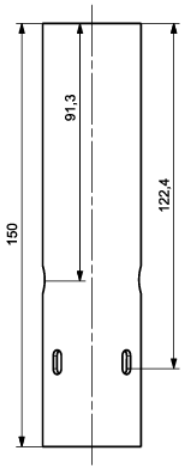


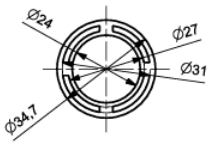

15,8
35
90
R6
43,5
2,2
2,3
5
Description: Sliding top for the servomotors
Part Name: Sliding Top
Materials: Generic PLA
Scale: 1:1
Rel. Date: 15.05.2025
Drawing no: 3
Rev. Date: 15.05.2025
Designer: Kiarash Zare Nasrabadi
SYDDANSK UNIVERSITET SØNDERBORG

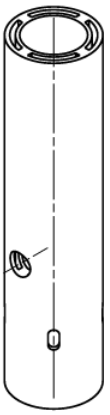

4,2
7,5
2
1,5
2,5
20
5
1,5
SECTION A-A
10
5,7
1
1
R2
10
Ø8
Ø6
A
A
8
8
Ø3
m = 0.5
z = 48
Description: Gear attached to the potentiometer
Part Name: Potentiometer Gear
Materials: Generic PLA
Scale: 2:1
Rel. Date: 28.05.2025
Drawing no: 4
Rev. Date: 28.05.2025
Designer: Kiarash Zare Nasrabadi
SYDDANSK UNIVERSITET SØNDERBORG


135°
1,1
30
1,5
72,8
37,4
3
SECTION A-A
B
10,6
6,9
1,6
7,1
R5
ϕA
3,2
DETAIL B
SCALE 1:1
R26,2
20,5
17,5°
28,8
7,1
A
A
22,9
78,8
30
R27,5
12
27,5
27,5
21,5
Ratio 1:1
Description: A holder for the LDR and the Solar Panels
Part Name: Solar Panel Arm
Materials: Generic PLA
Scale: 1:2
Rel. Date: 19.05.2025
Drawing no: 5
Rev. Date: 19.05.2025
Designer: Anne Cecilia Augustinsson
SYDDANSK UNIVERSITET SØNDERBORG


150
ϕ24
The holes are
custom design and
their distnace is arbitary.
In this case they are
oval but any shape works.
C
2,5
5,5
C
ϕ24
ϕ35
Ø31
Ø27
10
Ø35
20
10
Ø3
5
16
2
4,5
10
SECTION C-C
Ratio 1:2
Description: Upper piece of the stem with ball
Part Name: Stem Upper
Materials: Generic PLA
Scale: 1:1
Rel. Date: 22.05.2025
Drawing no: 6
Rev. Date: 22.05.2025
Designer: Kiarash Zare Nasrabadi
SYDDANSK UNIVERSITET SØNDERBORG



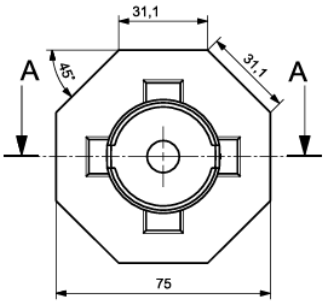


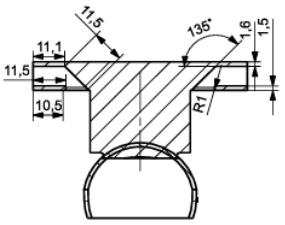


The holes are
custom design and
their distnace is arbitary.
In this case they are
oval but any shape works.

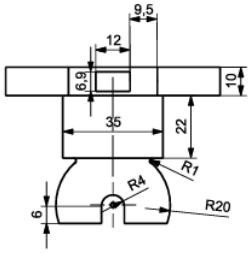
Description:	Lower piece of the stem
Part Name:	Stem Lower
Materials:	Generic PLA
Scale:	1:1
Rel. Date:	22.05.2025
Drawing no:	7
Rev. Date:	22.05.2025
Designer:	Kiarash Zare Nasrabadi

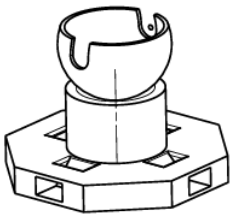
SYDDANSK UNIVERSITET SØNDERBORG





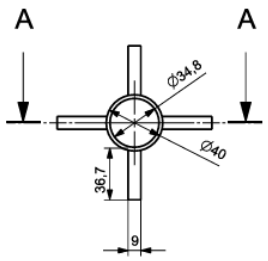
SECTION A-A

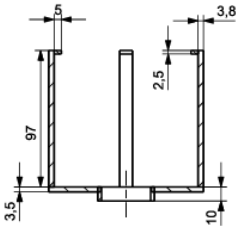




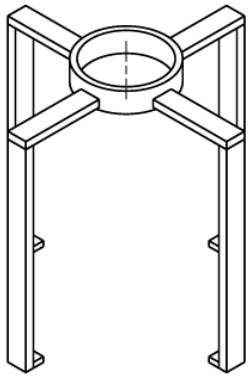
Description:	Socket and Holder for the Solar panel plates
Part Name:	Socket
Materials:	Generic PLA
Scale:	1:1
Rel. Date:	21.05.2025
Drawing no:	8
Rev. Date:	21.05.2025
Designer:	Anne Cecilia Augustinsson

SYDDANSK UNIVERSITET SØNDERBORG





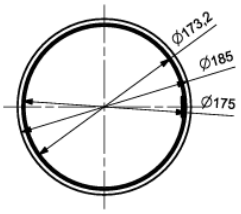
SECTION A-A

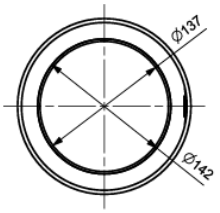


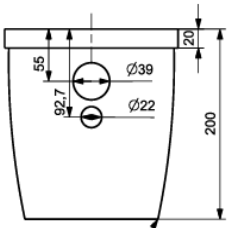
Ratio 1:1

Description:	A Holder to keep the servo motors in place
Part Name:	Servo motor Holder
Materials:	Generic PLA
Scale:	1:2
Rel. Date:	21.05.2025
Drawing no.	9
Rev. Date:	21.05.2025
Designer:	Kiarash Zare Nasrabadi

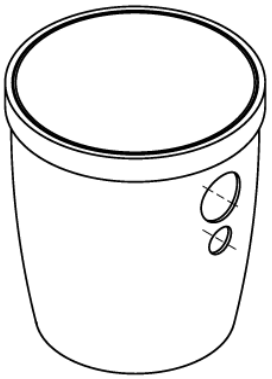
SYDDANSK UNIVERSITET SØNDERBORG







The Curve on the side is custom design.



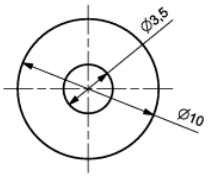
Ratio 1:2

Description:	Outer Design holding all components
Part Name:	Pot
Materials:	Generic PLA
Scale:	1:3
Rel. Date:	22.05.2025
Drawing no.	10
Rev. Date:	22.05.2025
Designer:	Kiarash Zare Nasrabadi

SYDDANSK UNIVERSITET SØNDERBORG

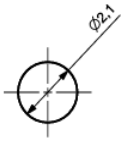


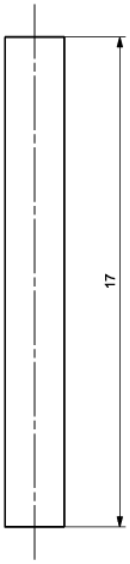


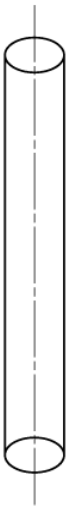


Description:	Pulleys for the Pulley system		
Part Name:	Pulleys		
Materials:	Generic PLA		
Scale:	5:1	Rel. Date:	19.05.2025
Drawing no.	11	Rev. Date:	19.05.2025
Designer:	Kiarash Zare Nasrabadi		

SYDDANSK UNIVERSITET SØNDERBORG

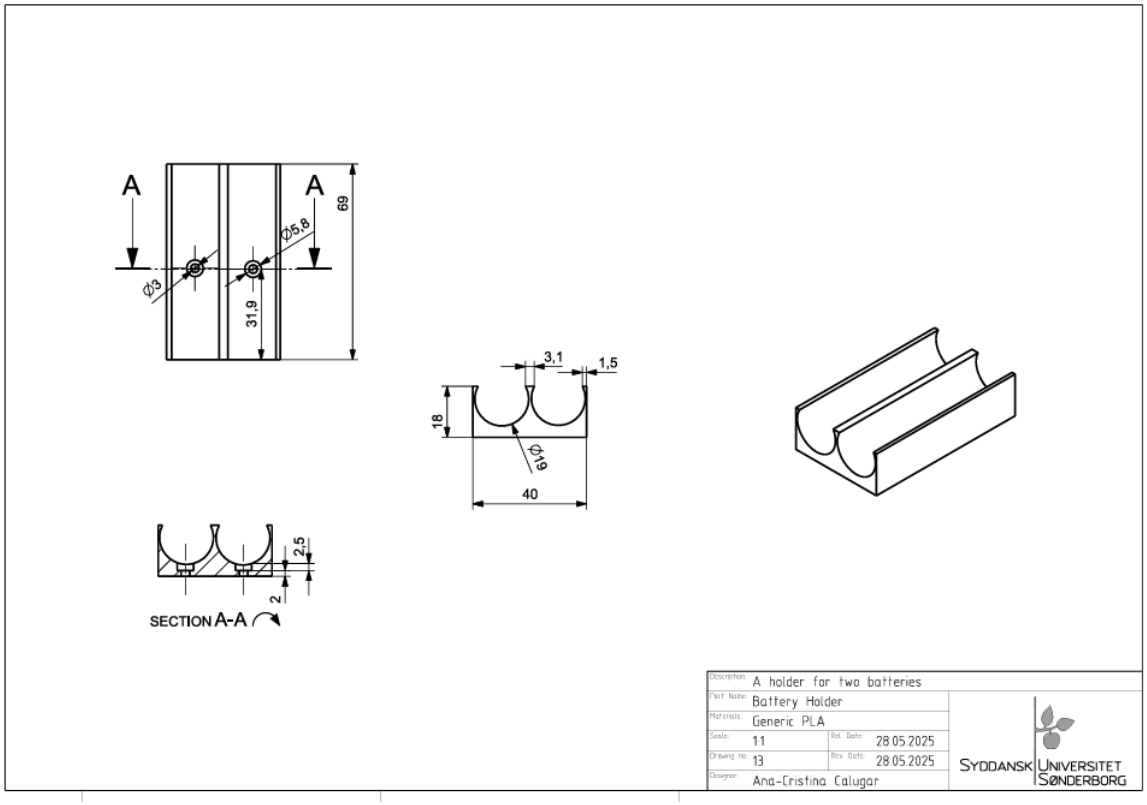


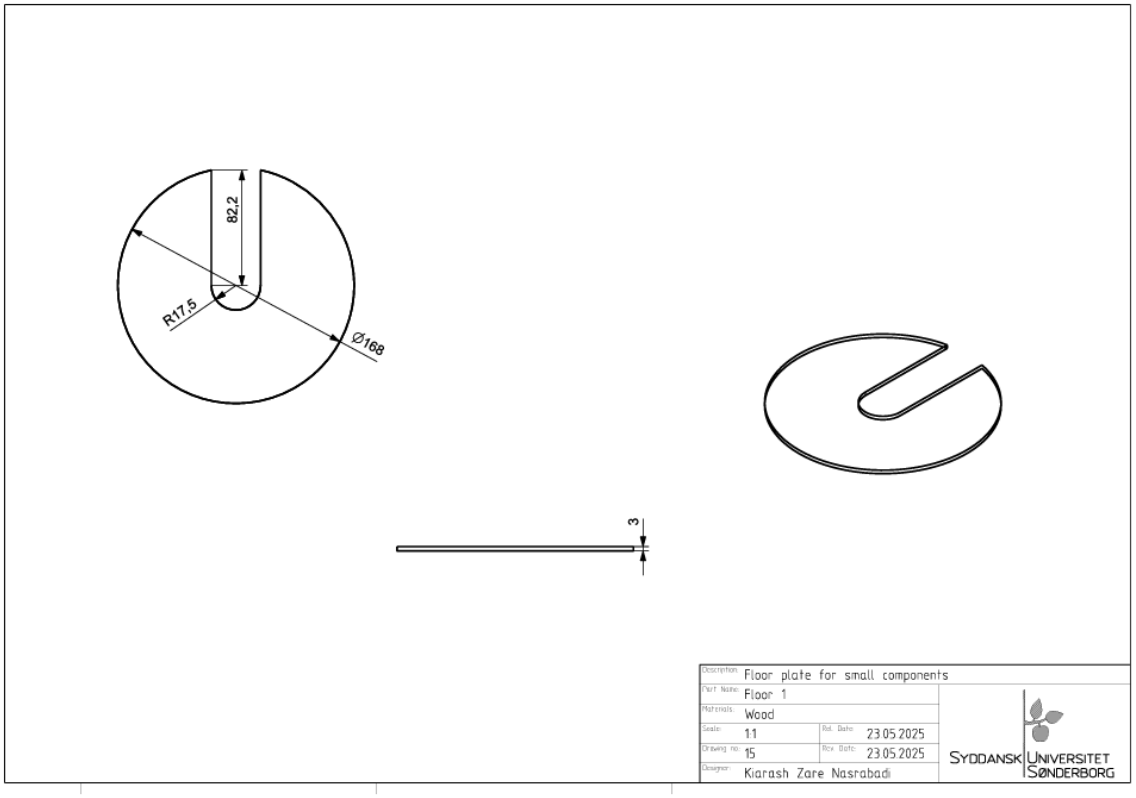


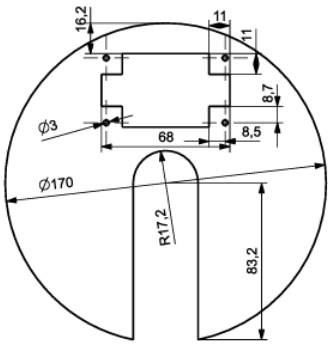


Description:	Shaft used to hold on the pulley		
Part Name:	Shaft		
Materials:	Generic PLA		
Scale:	10:1	Rel. Date:	21.05.2025
Drawing no.	12	Rev. Date:	21.05.2025
Designer:	Kiarash Zare Nasrabadi		

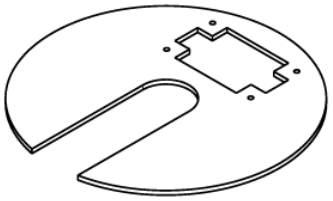
SYDDANSK UNIVERSITET SØNDERBORG


A
A
69
Ø5,8
Ø3
31,9
3,1
1,5
18
Ø19
40
2,5
2
SECTION A-A
Description: A holder for two batteries
Part Name: Battery Holder
Materials: Generic PLA
Scale: 1:1
Rel. Date: 28.05.2025
Drawing no.: 13
Rev. Date: 28.05.2025
Designer: Ana-Cristina Calugar
SYDDANSK UNIVERSITET SØNDERBORG


82,2
R17,5
Ø168
3
Description: Floor plate for small components
Part Name: Floor 1
Materials: Wood
Scale: 1:1
Rel. Date: 23.05.2025
Drawing no.: 15
Rev. Date: 23.05.2025
Designer: Kiarash Zare Nasrabadi
SYDDANSK UNIVERSITET SØNDERBORG

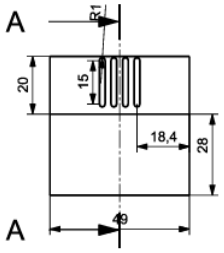


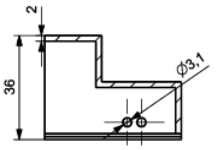




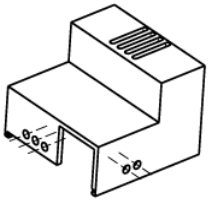
Description:	Floor plate for Nexion
Part Name:	Floor 2
Materials:	Wood
Scale:	1:1.5
Rel. Date:	24.05.2025
Drawing no.	16
Rev. Date:	24.05.2025
Designer:	Kiarash Zare Nasrabadi

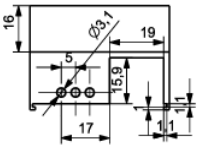
SYDDANSK UNIVERSITET SØNDERBORG





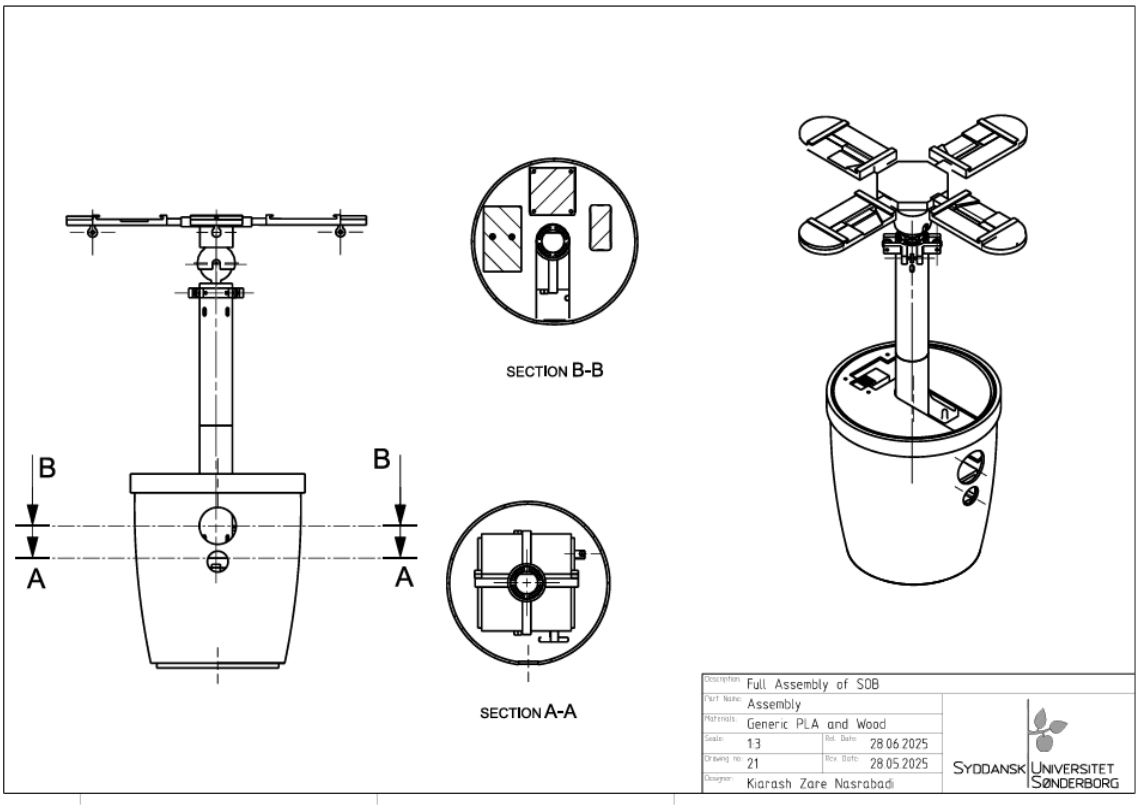
SECTION A-A

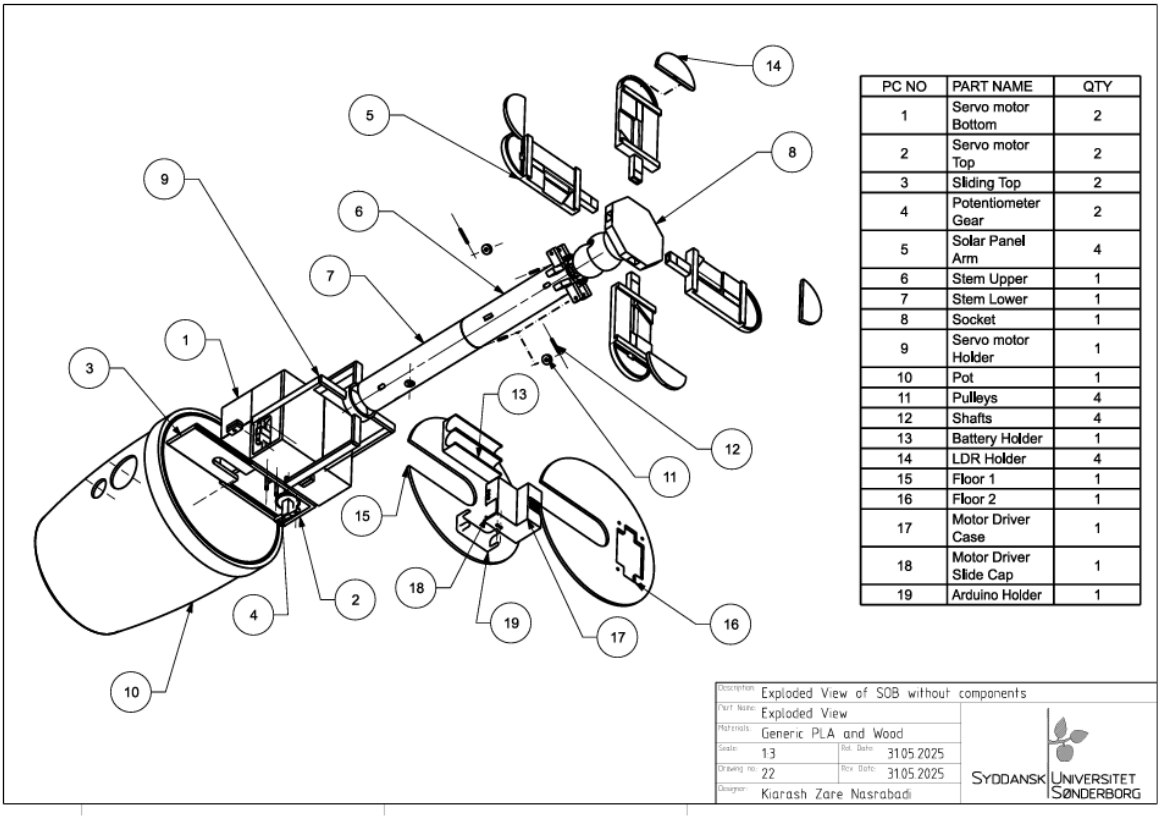




Description:	Case for the motor driver
Part Name:	Motor Driver Case
Materials:	Generic PLA
Scale:	1:1
Rel. Date:	15.11.2024
Drawing no.	17
Rev. Date:	15.11.2024
Designer:	Kiarash Zare Nasrabadi

SYDDANSK UNIVERSITET SØNDERBORG





PC NO	PART NAME	QTY
1	Servo motor Bottom	2
2	Servo motor Top	2
3	Sliding Top	2
4	Potentiometer Gear	2
5	Solar Panel Arm	4
6	Stem Upper	1
7	Stem Lower	1
8	Socket	1
9	Servo motor Holder	1
10	Pot	1
11	Pulleys	4
12	Shafts	4
13	Battery Holder	1
14	LDR Holder	4
15	Floor 1	1
16	Floor 2	1
17	Motor Driver Case	1
18	Motor Driver Slide Cap	1
19	Arduino Holder	1

8.2. Separate C Codes

8.2.1. Potentiometers + Servo Motors + LDRs + Motion Sensor C code

```
        /* Created: April/ May
 * Author : Ridhima */

     #include <avr/io.h>
       #include <util/delay.h>
       #include <avr/eeprom.h>
       #include <avr/interrupt.h>
       #include <stdio.h>
       #include "usart.h"

       // === Pin Definitions ===
       // Motor A control pins
       #define IN3 PD5  // Motor A IN3
       #define IN4 PD6  // Motor A IN4
       #define ENB PB1  // Motor A ENB (OC1A, D9)

       // Motor B control pins
       #define IN1 PD3  // Motor B IN1
       #define IN2 PD4  // Motor B IN2
       #define ENA PB2  // Motor B ENA (OC1B, D10)

       // Motion sensor input
       #define MOTION_PIN PD2  // Motion sensor input (INT0) for
interrupt

       //Analog input channels for ADC
       #define potA 0   // ADC0
       #define potB 5   // ADC5
       #define LDR1 1   // ADC1
       #define LDR2 2   // ADC2
       #define LDR3 3   // ADC3
       #define LDR4 4   // ADC4

       #define EEPROM_A_ADDR 0x00 // EEPROM address for Motor A's best
LDR average
       #define EEPROM_B_ADDR 0x04 // EEPROM address for Motor B's best
LDR average

       float angleA = 0.0f; // Current angle of Motor A (degrees)
       float angleB = 0.0f; // Current angle of Motor B (degrees)
```

```
        // === millis() timer setup ===
        volatile uint32_t milliseconds = 0;

        void timer1_init(void) {
            TCCR1B |= (1 << WGM12);                    // CTC mode
            TCCR1B |= (1 << CS11) | (1 << CS10);       // Prescaler 64
            OCR1A = 249;                               // 16MHz / 64 /
1000 - 1 = 249
            TIMSK1 |= (1 << OCIE1A);                   // Enable
Compare A Match interrupt
            sei();                                     // Enable
global interrupts
        }

        ISR(TIMER1_COMPA_vect) {
            milliseconds++;
        }

        uint32_t millis(void) {
            uint32_t ms;
            cli();
            ms = milliseconds;
            sei();
            return ms;
        }

        // === Motion Interrupt Setup ===
        void motion_interrupt_init(void) {
            EICRA |= (1 << ISC00); // Interrupt on any logical change
on INT0 (PD2)
            EIMSK |= (1 << INT0);  // Enable INT0 interrupt
            DDRD &= ~(1 << MOTION_PIN); // Set MOTION_PIN as input
            PORTD &= ~(1 << MOTION_PIN); // Disable pull-up (assuming
external pull-down or sensor handles it)
        }

        // === Motion Interrupt Handler ===
        ISR(INT0_vect) {
            if (PIND & (1 << MOTION_PIN)) {
                printf("MOTION: DETECTED at %lu ms\r\n", millis());
            } else {
                printf("MOTION: NONE at %lu ms\r\n", millis());
```

```
        }
    }

    // === ADC Read ===
    uint16_t analog_read(uint8_t channel) {
        ADMUX = (1 << REFS0) | (channel & 0x07);
        ADCSRA |= (1 << ADSC);
        while (ADCSRA & (1 << ADSC));
        return ADC;
    }

    // === PWM Init ===
    void pwm_init(void) {
        DDRB |= (1 << ENB) | (1 << ENA);
        TCCR1A = (1 << WGM10) | (1 << COM1A1) | (1 << COM1B1);
        TCCR1B |= (1 << WGM12);
        OCR1A = 0; OCR1B = 0;
    }



    // === Motor A Control ===
    void motorA_set_direction(uint8_t forward) {
        if (forward) { PORTD |= (1 << IN3); PORTD &= ~(1 << IN4); }
        else { PORTD &= ~(1 << IN3); PORTD |= (1 << IN4); }
    }
    void motorA_set_speed(uint8_t speed) { OCR1A = speed; }
    void motorA_stop(void) { OCR1A = 0; }

    // === Motor B Control ===
    void motorB_set_direction(uint8_t forward) {
        if (forward) { PORTD |= (1 << IN1); PORTD &= ~(1 << IN2); }
        else { PORTD &= ~(1 << IN1); PORTD |= (1 << IN2); }
    }
    void motorB_set_speed(uint8_t speed) { OCR1B = speed; }
    void motorB_stop(void) { OCR1B = 0; }



    // === LDR A/B Averages ===
    float ldrA_avg(void) {
        return (analog_read(LDR1) + analog_read(LDR2)) / 2.0f;
```

```
        }
        float ldrB_avg(void) {
            return (analog_read(LDR3) + analog_read(LDR4)) / 2.0f;
        }


        // === Rotate Motor Function ===
        void rotate_motor(const char* motor_name, const char* pot_name,
                          float target, float* current, uint8_t pot_pin,
                          void (*set_dir)(uint8_t), void
(*set_speed)(uint8_t), void (*stop)(),
                          float (*ldr_avg)()) {
            printf("[INFO] Rotating Motor %s to %.2f° using Pot
%s\r\n", motor_name, target, pot_name);
            while ((*current < target - 1.0f) || (*current > target +
1.0f)) {
                if (*current < target) {
                    printf("→ Motor %s: FORWARD\r\n", motor_name);
                    set_dir(1);
                } else {
                    printf("← Motor %s: REVERSE\r\n", motor_name);
                    set_dir(0);
                }
                set_speed(255);
                _delay_ms(200);
                *current = (analog_read(pot_pin) * 300.0f) / 1023.0f;
                printf("   ↳ Pot %s angle: %.2f°, LDR Avg: %.2f\r\n",
pot_name, *current, ldr_avg());
            }
            stop();
            printf("[DONE] Motor %s reached %.2f°\r\n", motor_name,
*current);
        }


        // === Main ===
        int main(void) {
            uart_init();
            io_redirect();

            DDRD |= (1 << IN3) | (1 << IN4) | (1 << IN1) | (1 << IN2);
            ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1);
            pwm_init();
```

```
            timer1_init();
            motion_interrupt_init(); // Initialize motion sensor
interrupt

            angleA = (analog_read(potA) * 300.0f) / 1023.0f;
            angleB = (analog_read(potB) * 300.0f) / 1023.0f;

            float bestA = eeprom_read_float((float*)EEPROM_A_ADDR);
            float bestB = eeprom_read_float((float*)EEPROM_B_ADDR);

            printf("Starting system...\r\n");
            for (int i = 5; i > 0; i--) {
                printf("%d...\r\n", i);
                _delay_ms(1000);
            }

            while (1) {
                // === Sweep Motor A ===
                float a_forward = angleA + 50.0f; if (a_forward >
300.0f) a_forward = 300.0f;
                rotate_motor("A", "A", a_forward, &angleA, potA,
motorA_set_direction, motorA_set_speed, motorA_stop, ldrA_avg);
                float avgA_f = ldrA_avg();

                float a_back = angleA - 100.0f; if (a_back < 0.0f)
a_back = 0.0f;
                rotate_motor("A", "A", a_back, &angleA, potA,
motorA_set_direction, motorA_set_speed, motorA_stop, ldrA_avg);
                float avgA_b = ldrA_avg();

                float best_avgA = (avgA_f > avgA_b) ? avgA_f : avgA_b;
                float best_angleA = (avgA_f > avgA_b) ? a_forward :
a_back;
                if (best_avgA > bestA * 1.05f) {
                    bestA = best_avgA;
                    eeprom_write_float((float*)EEPROM_A_ADDR, bestA);
                }
                rotate_motor("A", "A", best_angleA, &angleA, potA,
motorA_set_direction, motorA_set_speed, motorA_stop, ldrA_avg);

                // === Sweep Motor B ===
                float b_forward = angleB + 50.0f; if (b_forward >
300.0f) b_forward = 300.0f;
```

```
                rotate_motor("B", "B", b_forward, &angleB, potB,
motorB_set_direction, motorB_set_speed, motorB_stop, ldrB_avg);
                float avgB_f = ldrB_avg();

                float b_back = angleB - 100.0f; if (b_back < 0.0f)
b_back = 0.0f;
                rotate_motor("B", "B", b_back, &angleB, potB,
motorB_set_direction, motorB_set_speed, motorB_stop, ldrB_avg);
                float avgB_b = ldrB_avg();

                float best_avgB = (avgB_f > avgB_b) ? avgB_f : avgB_b;
                float best_angleB = (avgB_f > avgB_b) ? b_forward :
b_back;
                if (best_avgB > bestB * 1.05f) {
                    bestB = best_avgB;
                    eeprom_write_float((float*)EEPROM_B_ADDR, bestB);
                }
                rotate_motor("B", "B", best_angleB, &angleB, potB,
motorB_set_direction, motorB_set_speed, motorB_stop, ldrB_avg);

                _delay_ms(5000); // wait before next cycle
            }

            return 0;
        }
```

8.2.2. Speaker and Audio Playback Code

```

//TEST CODE FOR SPEAKER



//user-defined libraries
#include "pitches.h" //this for playing music from notes
#include <PCM.h> //this for playing encoded audio

//define audio pin
#define buzzer_pin 11 //sets D11 as audio pin
//this will be used for the "star wars" and "surprise" songs; the barking uses a different code

//for testing, we need to define a button pin
//later the music playing functionality will be triggered by the motion sensor
const int buttonPin = 2; //pin where button is connected


//variable for storing which sound is selected
int default_alarm; //could be 0 (barking), 1 (sw) or 2 (spr)

//we also define arrays we want to use
int melody[88];
int durations[88];
//make sure that both sw and spr arrays are the same length
//the size of these are based on that

```

```

//define the sw and spr melody[] and durations[] here
int melody_sw[]{
  NOTE_AS4, NOTE_AS4, NOTE_AS4,
  NOTE_F5, NOTE_C6,
  NOTE_AS5, NOTE_A5, NOTE_G5, NOTE_F6, NOTE_C6,
  NOTE_AS5, NOTE_A5, NOTE_G5, NOTE_F6, NOTE_C6,
  NOTE_AS5, NOTE_A5, NOTE_AS5, NOTE_G5, NOTE_C5, NOTE_C5, NOTE_C5,
  NOTE_F5, NOTE_C6,
  NOTE_AS5, NOTE_A5, NOTE_G5, NOTE_F6, NOTE_C6,

  NOTE_AS5, NOTE_A5, NOTE_G5, NOTE_F6, NOTE_C6,
  NOTE_AS5, NOTE_A5, NOTE_AS5, NOTE_G5, NOTE_C5, NOTE_C5,
  NOTE_D5, NOTE_D5, NOTE_AS5, NOTE_A5, NOTE_G5, NOTE_F5,
  NOTE_F5, NOTE_G5, NOTE_A5, NOTE_G5, NOTE_D5, NOTE_E5, NOTE_C5, NOTE_C5,
  NOTE_D5, NOTE_D5, NOTE_AS5, NOTE_A5, NOTE_G5, NOTE_F5,

  NOTE_C6, NOTE_G5, NOTE_G5, REST, NOTE_C5,
  NOTE_D5, NOTE_D5, NOTE_AS5, NOTE_A5, NOTE_G5, NOTE_F5,
  NOTE_F5, NOTE_G5, NOTE_A5, NOTE_G5, NOTE_D5, NOTE_E5, NOTE_C6, NOTE_C6,
  NOTE_F6, NOTE_DS6, NOTE_CS6, NOTE_C6, NOTE_AS5, NOTE_GS5, NOTE_G5, NOTE_F5,
  NOTE_C6
};

```

```
52
53  int durations_sw[]{
54    8, 8, 8,
55    2, 2,
56    8, 8, 8, 2, 4,
57    8, 8, 8, 2, 4,
58    8, 8, 8, 2, 8, 8, 8,
59    2, 2,
60    8, 8, 8, 2, 4,
61
62    8, 8, 8, 2, 4,
63    8, 8, 8, 2, 8, 16,
64    4, 8, 8, 8, 8, 8,
65    8, 8, 8, 4, 8, 4, 8, 16,
66    4, 8, 8, 8, 8, 8,
67
68    8, 16, 2, 8, 8,
69    4, 8, 8, 8, 8, 8,
70    8, 8, 8, 4, 8, 4, 8, 16,
71    4, 8, 4, 8, 4, 8, 4, 8,
72    1
73  };
74
75  int melody_spr[]{
76    NOTE_A4, REST, NOTE_B4, REST, NOTE_C5, REST, NOTE_A4, REST,
77    NOTE_D5, REST, NOTE_E5, REST, NOTE_D5, REST,
78
79    NOTE_G4, NOTE_A4, NOTE_C5, NOTE_A4, NOTE_E5, NOTE_E5, REST,
80    NOTE_D5, REST,
81
82    NOTE_G4, NOTE_A4, NOTE_C5, NOTE_A4, NOTE_D5, NOTE_D5, REST,
83    NOTE_C5, REST, NOTE_B4, NOTE_A4, REST,
84
85    NOTE_G4, NOTE_A4, NOTE_C5, NOTE_A4, NOTE_C5, NOTE_D5, REST,
86    NOTE_B4, NOTE_A4, NOTE_G4, REST, NOTE_G4, REST, NOTE_D5, REST, NOTE_C5, REST,
```

```
87
88      NOTE_G4, NOTE_A4, NOTE_C5, NOTE_A4, NOTE_E5, NOTE_E5, REST,
89      NOTE_D5, REST,
90
91      NOTE_G4, NOTE_A4, NOTE_C5, NOTE_A4, NOTE_G5, NOTE_B4, REST,
92      NOTE_C5, REST, NOTE_B4, NOTE_A4, REST,
93
94      NOTE_G4, NOTE_A4, NOTE_C5, NOTE_A4, NOTE_C5, NOTE_D5, REST,
95      NOTE_B4, NOTE_A4, NOTE_G4, REST, NOTE_G4, REST, NOTE_D5, REST
96    };
97
98    int durations_spr[]{
99      8, 8, 8, 8, 8, 8, 8, 4,
100     8, 8, 8, 8, 2, 2,
101
102     8, 8, 8, 8, 2, 8, 8,
103     2, 8,
104
105     8, 8, 8, 8, 2, 8, 8,
106     4, 8, 8, 8, 8,
107
108     8, 8, 8, 8, 2, 8, 8,
109     2, 8, 4, 8, 8, 8, 8, 8, 1, 4,
110
111     8, 8, 8, 8, 2, 8, 8,
112     2, 8,
113
114     8, 8, 8, 8, 2, 8, 8,
115     2, 8, 8, 8, 8,
116
117     8, 8, 8, 8, 2, 8, 8,
118     4, 8, 3, 8, 8, 8, 8, 8
119   };
120
121
```

```
121
122   //now for the barking:
123   const unsigned char sample[] PROGMEM = {
124     //here goes the encoded barking --- for practical reasons, it will only be included once the rest of the code is written
125   }
126
127
128
129
130
131   //NEXTION related setup
132   //declare objects we are reading from the display:
133   //format: function name = function(page number, ID, "name") --- these are already changed to final pg.no. and IDs!!!
134   NexRadio r0 = NexRadio(2, 5, "r0");
135   NexRadio r1 = NexRadio(2, 6, "r1");
136   NexRadio r2 = NexRadio(2, 7, "r2");
137
138   //declare touch event objects to the touch event list:
139   //format: &<name>
140   NexTouch *nex_listen_list[] =
141   {
142     &r0, //radio checkbox added
143     &r1,
144     &r2,
145     NULL //string terminated
146   };
147
148
```

```
148
149  //touch events:
150  //each touch event will trigger one of the following sections
151
152  void r0PushCallback(void *ptr) //press event for radio checkbox r0
153  {
154    //code for setting the default sound to WOOF WOOF
155    int value_r0 = Serial.print("get r0.val"); //reads the value of r0 --- could be 0(not pressed) or 1 (pressed)
156
157    Serial.write(0xff);
158    Serial.write(0xff);
159    Serial.write(0xff);
160  }
161
162  void r1PushCallback(void *ptr) //press event for radio checkbox r1
163  {
164    //code for setting the default sound to STAR WARS
165    int value_r1 = Serial.print("get r1.val");
166
167    Serial.write(0xff);
168    Serial.write(0xff);
169    Serial.write(0xff);
170  }
171
172  void r2PushCallback(void *ptr) //press event for radio checkbox r2
173  {
174    //code for setting the default sound to SURPRISE
175    int value_r2 = Serial.print("get r2.val");
176
177    Serial.write(0xff);
178    Serial.write(0xff);
179    Serial.write(0xff);
180  }
181
```

```
182
183
184  void setup() {
185
186    Serial.begin(9600);
187
188    //register the event callback functions of each touch event:
189    r0.attachPush(r0PushCallback); //radio checkbox press
190    r1.attachPush(r1PushCallback);
191    r2.attachPush(r2PushCallback);
192
193    pinMode(buzzer_pin, OUTPUT); //setting our previously defined D11 as audio output pin
194    pinMode(buttonPin, INPUT_PULLUP); //enable internal pull-up resistor for button --- only needed for testing
195
196  }
197
198  void loop() {
199
200    //checking the list of touch events we defined to know if any of them just happened
201    //and then execute the corresponding instructions
202    nexLoop(nex_listen_list); //in this case, the corresponding instruction would be assigning a value to default_alarm
203
204
205    if(value_r0 == 0){ //if it's NOT r0 selected
206
207      if(value_r1 == 1){ //it can be r1 selected if r1=1
208        default_alarm = 1;
209      }
210      else{ //the remaining one case is if r2 is selected
211        default_alarm = 2;
212      }
213
214      //here goes function no. 1 --- if r1 or r2 was pressed, then the melody[] and durations[] get populated
215      void melodyFunction (default_alarm, melody_sw, durations_sw, melody_spr, durations_spr, melody, durations, sizeof(durations)/sizeof(int));
```

```
    //here goes function no. 1 --- if r1 or r2 was pressed, then the melody[] and durations[] get populated
    void melodyFunction (default_alarm, melody_sw, durations_sw, melody_spr, durations_spr, melody, durations, sizeof(durations)/sizeof(int));

  }

  //as the barking is based on a different playback type, this should be enough for now
  else{ //when r0 is selected
    default_alarm = 0;
  }


  //inside a button press loop (later: motion sensor triggered loop)

  for(digitalRead(buttonPin) == LOW){ //checks if button is pressed --- low meaning pressed

    if(default_alarm != 0){ //so if r1 or r2 is selected
      //call function no.2
      playMusic(melody, durations, sizeof(durations)/sizeof(int));
    }

    else{
      //call other function for barking playback --- function included in the PCM library
      startPlayback(sample, sizeof(sample));
    }

  }
}
```

```


//////////  F U N C T I O N S  //////////

//2 functions needed:
    // 1) for modifying melody[] and durations[] depending on selected default sound ---> returns the melody[] and durations[] to main
    // 2) for actually playing the melody when sensor is triggered


//function no.1: --- for populating arrays
void melodyFunction (int variable, int *source1, int *source2, int *source3, int *source4, int *destination1, int *destination2, int size){


  if(variable == 1){ //if r1 is selected
    for (int i = 0; i < size; i++){
      destination1[i] = source1[i];
      destination2[i] = source2[i];
      }
    }

  if(variable == 2){ //if r2 is selected
    for (int i = 0; i < size; i++){
      destination1[i] = source3[i];
      destination2[i] = source4[i];
      }
    }


}
```

```
273
274  //function no.2: --- for playing notes based on arrays
275  void playMusic(int *melody, int *durations, int size){
276
277    int size = sizeof(durations)/sizeof(int);
278
279    for (int note = 0; note < size; note++){
280      //to calculate the note duration, take one second divided by the note type.
281      //e.g. quarter note = 1000 / 4, eighth note = 1000/8, etc.
282      int duration = 1000 / durations[note];
283      tone(BUZZER_PIN, melody[note], duration);
284
285      //to distinguish the notes, set a minimum time between them.
286      //the note's duration + 30% seems to work well:
287      int pauseBetweenNotes = duration * 1.30;
288      delay(pauseBetweenNotes);
289
290      //stop the tone playing:
291      noTone(BUZZER_PIN);
292    }
293
294  }
295
296  //end of code
```

8.2.3. Arduino to Nextion Display Communication

```
#define BAUD 9600   // Set baud rate for USART communication

#include <stdio.h>// For sprintf and snprintf
#include <util/delay.h>// For _delay_ms()
#include <avr/io.h>// For direct register access
#include <string.h> // For string functions
#include "usart.h" // Custom USART library

#define A6 6 // Define A6 as ADC channel 6
#define A7 7  // Define A7 as ADC channel 7

#define MAX_EVENTS 10 // Max number of motion events to track
#define SECONDS_IN_24H 86400UL // Number of seconds in 24 hours

// ===== STRUCT DEFINITIONS =====

typedef struct {
    uint8_t hours;
    uint8_t minutes;
    uint8_t seconds;
} Clock;  // Structure for time tracking

typedef struct {
    Clock time;
    uint32_t created_at_seconds;
    uint8_t active;
} MotionEvent; // Structure to store motion event data

MotionEvent events[MAX_EVENTS];// Array to store motion events

// ===== INITIALIZATION FUNCTIONS =====

void adc_init() {
    ADMUX = (1 << REFS0);// AVcc as voltage reference
    ADCSRA = (1 << ADEN) // Enable ADC
           | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);  // Prescaler = 128
}


// ===== CLOCK FUNCTIONS =====

void incrementClock(Clock* clk) {
```

```
void incrementClock(Clock* clk) {
    clk->seconds++; // Increment seconds
    if (clk->seconds >= 60) {
        clk->seconds = 0;
        clk->minutes++; // Increment minutes
        if (clk->minutes >= 60) {
            clk->minutes = 0;
            clk->hours++; // Increment hours
        }
        if (clk->hours >= 24) {
            clk->hours = 0; // Reset after 24 hours
        }
    }
}

uint32_t getSeconds(const Clock* clk) {
    return clk->hours * 3600UL + clk->minutes * 60UL + clk->seconds;
} // Convert time to total seconds

// ===== ANALOG READ FUNCTIONS =====

int analogRead(uint8_t channel) {
    ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);  // Select ADC channel
    ADCSRA |= (1 << ADSC); // Start conversion
    while (ADCSRA & (1 << ADSC)); // Wait for conversion to finish
    return ADC;   // Return result
}

int analogRead_A6() { return analogRead(A6); }  // Read from A6 (ADC6)
int analogRead_A7() { return analogRead(A7); }  // Read from A7 (ADC7)

// ===== NEXTION DISPLAY COMMUNICATION =====

void sendToNextion(const char* cmd) {
    for (size_t i = 0; cmd[i] != '\0'; i++) {
        putchar(cmd[i]);  // Send command one character at a time
    }
    putchar(0xFF);  // Termination bytes required by Nextion
    putchar(0xFF);
    putchar(0xFF);
```

```
    putchar(0xFF);
}

// ===== UTILITY FUNCTIONS =====

float constrain(float x, float a, float b) {
    if (x < a) return a;
    if (x > b) return b;
    return x;
}

}

void addMotionEvent(const Clock* now, uint32_t nowSeconds) {
    for (uint8_t i = 0; i < MAX_EVENTS; i++) {
        if (!events[i].active) {  // Find an inactive slot
            events[i].time = *now; // Store timestamp
            events[i].created_at_seconds = nowSeconds;
            events[i].active = 1;

            char cmd[40];
            snprintf(cmd, sizeof(cmd), "Clock.t%d.txt=\"%02d.%02d.%02d\"",
                     i + 1, now->hours, now->minutes, now->seconds);
            sendToNextion(cmd);   // Display event on Nextion screen
            break;
        }
    }
}

void cleanupOldEvents(uint32_t nowSeconds) {
    for (uint8_t i = 0; i < MAX_EVENTS; i++) {
        if (events[i].active && (nowSeconds - events[i].created_at_seconds) >= SECONDS_IN_24H) {
            events[i].active = 0; // Deactivate event older than 24h

            char cmd[30];
            snprintf(cmd, sizeof(cmd),"Clock.t0.txt=\"\"", i + 1);
            sendToNextion(cmd);// Clear event from screen
        }
    }
}

```

```
// ===== MAIN PROGRAM =====

int main() {
    uart_init();  // Initialize UART (serial)
    io_redirect();  // Redirect stdout to UART
    adc_init(); // Initialize ADC
    motion_pin_init(); // Configure motion sensor pin

    Clock clock = {0, 0, 0};   // Start clock at 00:00:00

    int vin1 = 0, vin2 = 0;  // ADC readings
    float vout1, vout2, avg_vout, avg_perc;
    const float R1 = 110000.0; // Resistor 1 for voltage divider
    const float R2 = 110000.0; // Resistor 2
    const float ratio = R2 / (R1 + R2); // Voltage divider ratio = 0.5

    char buffer[50]; // Buffer for display strings
    uint8_t lastMotionState = 0; // For motion edge detection

    while (1) {
        // ==== Battery voltage measurement ====
        vin1 = analogRead_A6();// Read ADC6 (A6)
        vin2 = analogRead_A7(); // Read ADC7 (A7)
        vout1 = vin1 * (5.0 / 1023.0) / ratio;  // Convert to actual voltage
        vout2 = vin2 * (5.0 / 1023.0) / ratio;
        avg_vout = (vout1 + vout2) / 2.0;// Average the two voltages

        //When the one battery is fully charged its maximum Voltage would be 4.2 and 3.0 when is the minimum
        avg_perc = (avg_vout - 3.0) * (100.0 / (4.2 - 3.0)); // Convert to %
        avg_perc = constrain(avg_perc, 0, 100); // Clamp between 0-100%

        snprintf(buffer, sizeof(buffer), "Battery.n0.val=%d", (int)avg_perc);
        sendToNextion(buffer); // Update battery % on display

        snprintf(buffer, sizeof(buffer), "Battery.n1.val=%d", (int)(avg_vout * 1000));
        sendToNextion(buffer); // Update voltage (in mV) on display

        // ==== Clock update ====
        incrementClock(&clock);                // Increment time by 1s
        snprintf(buffer, sizeof(buffer), "Clock.t0.txt=\"%02d:%02d:%02d\"",
                 clock.hours, clock.minutes, clock.seconds);
        sendToNextion(buffer);                 // Update time on display
        snprintf(buffer, sizeof(buffer), "Clock.t0.txt=\"%02d:%02d:%02d\"",
                 clock.hours, clock.minutes, clock.seconds);
        sendToNextion(buffer);                  // Update time on display

        uint32_t nowSeconds = getSeconds(&clock);// Current time in seconds

        // ==== Motion Detection ====
        uint8_t currentMotionState = motionDetected();  // Check motion sensor; chang
        if (currentMotionState && !lastMotionState) {
            addMotionEvent(&clock, nowSeconds);// Log event on rising edge
        }
        lastMotionState = currentMotionState;

        cleanupOldEvents(nowSeconds); // Remove expired events

        _delay_ms(1000);
    }

    return 0;
}
```